



AWS AI Security Agent

Threat Model Report

JuiceShop-Threat-Model

August 13, 2026

Table of Contents

Introduction	6
System Overview	7
Configuration	20
Threats (83)	21
The external webhook receiver operator, given that host-metadata is sent in...	27
A network attacker on the egress path, given that the outbound webhook call...	28
An attacker who can modify docker-compose.yml can alter REDIS_URL or inject...	29
An attacker with write access to the repository or build cache, given no image...	30
An attacker exploiting known vulnerabilities in MongoDB 4.4.29, given the...	31
Any container on the juice-shop-net bridge network, given MongoDB runs without...	32
Any container on the juice-shop-net bridge network, given Redis runs without...	33
Any user with access to the container image, given the Docker build context...	34
An attacker with server filesystem access, gained through a separate backend...	35
An end user can extract challenge hints and hacking-instructor tutorial steps...	36
An end user can modify or disable client-side route guard logic in the...	37
An unauthenticated end user can inspect the delivered JavaScript bundle to...	38
A network attacker (on-path), when TLS is not enforced at deployment, can...	39
An anonymous end user performing SQL injection attacks against...	40
A network-adjacent attacker positioned on the same network segment can...	41
An authenticated end user holding any valid JWT can submit POST /api/Feedbacks...	42
Any user with source code access who obtains the hardcoded RSA private key from...	43

An anonymous end user can send PUT requests to /api/Products/:id to modify...	44
An anonymous end user can inject a UNION SELECT against sqlite_master through...	45
An anonymous end user can craft a UNION SELECT payload in the search query...	46
An anonymous end user can inject arbitrary SQL statements through the q...	47
Attacker who achieved directory traversal write or Node.js process compromise...	48
End User (Browser) can browse serve-index directory listings at /ftp,...	49
End User (Browser) can access sensitive files via file-serving endpoints...	50
Network-positioned attacker who can intercept network traffic between user and...	51
End User (Browser) can request /support/logs/ to enumerate and download...	52
End User (Browser) who knows that forward-slash check can be bypassed with...	53
End User (Browser) can request /encryptionkeys/premium.key without...	54
End User (Browser) can request /encryptionkeys/jwt.pub directly without...	55
End User (Browser) who knows the null-byte injection technique (%2500) and the...	56
An unauthenticated WebSocket client who knows expected payload patterns can...	57
A network-level attacker in a shared lab environment can intercept unencrypted...	58
A late-connecting WebSocket client can receive the full replay of cached...	59
Any connected WebSocket client can passively receive broadcast 'challenge...	60
A malicious participant who can connect to the unauthenticated WebSocket...	61
An external attacker with network access to port 3000 can connect to the...	62
An authenticated end-user can store an attacker-controlled URL in the...	63
An authenticated end-user can perform SSRF requests through the profile image...	64
A network-adjacent attacker on the same LAN can sniff unencrypted HTTP traffic...	65
An authenticated end-user controlling an external server can serve arbitrary...	66
An external attacker with a stolen cookie token can invoke /profile/image/url...	67

An authenticated end-user can supply a malicious URL that fails to fetch,...	68
An authenticated end-user with a valid session cookie can supply a URL...	69
An end user, given that user messages are unfiltered and JWT identity is...	70
An anonymous internet attacker, given that no rate limiting or authentication...	71
The external LLM API, given that no TLS enforcement is verified in source and...	72
An anonymous internet attacker, given that security.decode uses jws.decode...	73
An end user, given that the getProductReviews tool uses MongoDB \$where with...	74
An end user can craft chat messages that instruct the LLM to reveal its system...	75
An end user can craft adversarial chat messages that override the system prompt...	76
An anonymous internet attacker, given that the /rest/chat endpoint has no...	77
An anonymous internet attacker, given that no authentication is enforced and no...	78
A network-positioned attacker, given that the application defaults to HTTP...	79
An anonymous internet attacker, given that parsed XML with resolved entities is...	80
An anonymous internet attacker, given that checkFileType and checkUploadSize...	81
An anonymous internet attacker, given that YAML parsing uses js-yaml.load (not...	82
An anonymous internet attacker, given that ZIP extraction uses an insufficient...	83
An anonymous internet attacker, given that the XML parser is configured with...	84
An anonymous internet attacker, given that the /file-upload endpoint has no...	85
An attacker with database read access via SQL injection can perform offline...	86
A network-level attacker in MITM position, given application uses HTTP without...	87
An external attacker, given no rate limiting or account lockout exists on...	88
An attacker exploiting XSS in the Juice Shop frontend, given the token is...	89
An external attacker with access to public source code, given the RSA private...	90
A network-level attacker, given application deployed with default HTTP...	91

An end-user-via-browser can inject SQL payload into the email field which is...	92
An unresponsive external webhook receiver can cause the outbound fetch in...	93
An external attacker who knows the webhook URL can send crafted POST requests...	94
An attacker with environment variable write access, given that webhook-url has...	95
An operator with Docker socket access can build and deploy a tampered container...	96
A malicious third-party website, exploiting the fully open CORS policy...	97
An anonymous end user can inject computationally expensive SQL statements...	98
An authenticated end user holding any valid JWT can send POST/PUT requests to...	99
An external attacker can open thousands of concurrent unauthenticated WebSocket...	100
A malicious training participant can deny triggering challenge solves since no...	101
A malicious WebSocket client can send malformed or oversized payloads through...	102
An authenticated end-user can craft a URL with a manipulated extension or path...	103
An authenticated end-user can supply URLs pointing to extremely large or...	104
The external LLM API, given that full user message history is forwarded on...	105
The external LLM API, given that the AI SDK executes tool calls without...	106
An anonymous internet attacker, given that ZIP extraction writes to the server...	107
An authenticated end user can modify the basket ID in sessionStorage or API...	108
An end user or attacker using compromised credentials can perform account...	109

Introduction

AWS Security Agent performed a threat model analysis of **JuiceShop-Threat-Model** (Job ID: tj-2f0cfb-dd-10c2-42a3-a860-e27a82c11f6e) on August 13, 2026, deploying specialized AI agents to discover the system architecture, trace data flows, and enumerate threats using STRIDE methodology. **83** threat(s) were identified: **66** high severity, **17** medium severity, and **0** low severity. Full details are available in the Threats section.

Understanding Severity Levels

High Threats that pose an immediate and significant risk to the system. Exploitation is likely and impact is severe.

Medium Threats with moderate risk. Exploitation requires specific conditions but impact could be significant.

Low Threats with limited risk. Exploitation is unlikely or impact is minimal.

Threat Status

Tracks the review workflow state. Identified threats require review. Mark threats as Resolved when mitigated, or Resolved (Not Useful) when determined to be inapplicable to your environment.

STRIDE Categories

Each threat is classified using the STRIDE methodology, an industry-standard framework for categorizing security threats:

Spoofing — Impersonating a user, system, or component to gain unauthorized access.

Tampering — Unauthorized modification of data or code, compromising integrity.

Repudiation — Ability to deny performing an action when there is no way to prove otherwise.

Information Disclosure — Exposure of sensitive data to unauthorized parties.

Denial of Service — Disrupting availability by overwhelming or crashing a system.

Elevation of Privilege — Gaining higher access rights than intended.

System Overview

OWASP Juice Shop

An intentionally vulnerable e-commerce web application designed for security training, awareness demonstrations, and capture-the-flag competitions.

Purpose

OWASP Juice Shop simulates an online juice store that contains deliberate security flaws spanning the OWASP Top 10 and beyond. It provides a safe, legal environment for security professionals, penetration testing students, and developers to practice identifying and exploiting web application vulnerabilities across authentication, authorization, injection, file handling, and more.

The application is built as a comprehensive security training platform that can be used in instructor-led workshops, self-paced learning, and competitive CTF events. Each vulnerability is presented as a "challenge" that participants must discover and exploit, with a scoring system that tracks progress and optionally reports solutions to external webhooks.

The primary consumers are security trainers conducting workshops, students learning application security, and developers building awareness of common vulnerability patterns. The application is typically deployed locally or in isolated lab environments. The entire codebase—including its vulnerabilities—is open source and intended to be explored, attacked, and learned from.

Capabilities

- Authenticate users via email/password login with JWT-based session management
- Search, browse, and purchase products through a simulated e-commerce storefront
- Upload files (complaints with attachments) that are processed server-side via ZIP extraction, XML parsing, and YAML deserialization
- Interact with an AI-powered chatbot backed by an external LLM with tool-calling capabilities for product search, order lookup, and coupon generation

- Serve static files from multiple directories including an FTP-like endpoint, encryption keys, and support logs
- Update user profiles including setting a profile image via URL
- Track and verify security challenge solutions with real-time WebSocket notifications and optional external webhook reporting
- Deploy via Docker Compose or Terraform with intentionally insecure infrastructure configurations as additional challenge material

Architecture

OWASP Juice Shop follows a monolithic single-page application architecture. An Angular frontend compiled into static JavaScript bundles communicates with an Express.js backend over HTTP. The backend provides RESTful APIs, file serving, WebSocket events via Socket.IO, and an AI chatbot integration. All components run within a single Node.js process backed by an embedded SQLite database for persistent storage and MarsDB for in-memory order data.

The backend implements intentionally weak security controls: JWT signing uses a hardcoded RSA private key, SQL queries use string interpolation rather than parameterized statements, file uploads are parsed without sanitization, and several endpoints lack authentication middleware entirely. These weaknesses are deliberate design decisions that form the basis of the security challenges.

Infrastructure-as-code definitions (Docker Compose and Terraform) package the application for deployment. The Docker Compose configuration defines a multi-container stack with unauthenticated Redis and MongoDB services. The Terraform configuration includes intentionally leaked credentials and private keys as additional CTF challenges. Neither is intended for production use.

Components

Name	Purpose	Key Interactions

juice-shop	Monolithic intentionally-vulnerable web application comprising frontend, backend, and infrastructure components all serving the unified OWASP Juice Shop security training platform	Serves compiled Angular frontend to browsers via Express static middleware; receives HTTP requests from browser clients on REST API endpoints; reads/writes to embedded SQLite database via Sequelize; accepts WebSocket connections from browsers via Socket.IO; makes outbound HTTP requests to external LLM API and SSRF targets; sends challenge solution notifications to external webhook URLs; packages into Docker container images via Docker Compose/Terraform
------------	---	--

Trust Boundaries

Since the system is a monolithic application, the trust boundaries exist at the interfaces between the application and external actors:

Browser-to-Server Boundary: The Angular SPA running in the user's browser communicates with the Express backend over HTTP. All user-supplied input (credentials, search queries, file uploads, URLs, chat messages, WebSocket events) crosses this boundary. Controls at this crossing are intentionally weak or absent: JWT authentication is applied inconsistently across endpoints, CORS allows all origins, no TLS is enforced at the application layer, and no CSRF protection exists.

Server-to-External Services Boundary: The backend makes outbound requests to external systems including the LLM API (authenticated via LLM_API_KEY), arbitrary URLs supplied by users (SSRF vector), and the solutions webhook endpoint. Controls include API key authentication for the LLM but no URL validation for SSRF targets and no authentication on webhook calls.

Application-to-Filesystem/Database Boundary: The Express process reads and writes to the local SQLite database and filesystem. User-controlled input (SQL queries, filenames, ZIP contents, XML with external entities) crosses from the request-handling context into database queries and filesystem operations. Controls are intentionally absent: no parameterized queries on search, no path validation on file serving, and external entity resolution is enabled in XML parsing.

Data Flows

User authentication via frontend to backend REST API

1. The end user enters email and password into the Angular login form. The LoginComponent collects these values in memory and passes them to UserService.login(). No data leaves the browser at this stage.

2. **[Crosses browser-to-server boundary]** The Angular HttpClient sends an HTTP POST to /rest/user/login with a JSON body containing {email, password}. The RequestInterceptor attaches a Bearer token header if one exists in localStorage (none on initial login). Transport is plaintext HTTP by default. No TLS enforcement exists in the application. **Controls:** None — credentials travel in cleartext.

3. **[Crosses application-to-database boundary]** The backend route handler in routes/login.ts executes a raw SQL SELECT query via models.sequelize.query(). The email is interpolated directly into the query string; the password is MD5-hashed via security.hash() before interpolation. No parameterized queries are used. The query runs in-process against the SQLite file. **Controls:** None — intentionally vulnerable to SQL injection.

4. Upon successful authentication, the backend generates an RS256 JWT using a hardcoded private key via security.authorize(), encoding user ID, email, role, and basket ID. The token (6-hour expiry) is returned in the HTTP response. The frontend stores the JWT in both localStorage and a cookie (without HttpOnly or Secure flags).

Malicious file upload with XML/XXE parsing

1. **[Crosses browser-to-server boundary]** The Angular complaint component sends a multipart HTTP POST to /file-upload carrying the uploaded file (ZIP/XML/YAML) in the 'file' field. A Bearer JWT token is attached in the Authorization header by ng2-file-upload configuration. Transport is plaintext HTTP. No CSRF protection exists; CORS allows all origins. **Controls:** JWT is attached but not verified by the backend endpoint; no authentication middleware guards /file-upload.

2. **[Crosses request-handling to file-processing boundary]** Express middleware passes `req.file.buffer` (attacker-controlled file content) to `handleZipFileUpload`, `handleXmlUpload`, and `handleYamlUpload` in sequence within `routes/fileUpload.ts`. No authentication context is verified. No sanitization is applied before parsing. ZIP extraction writes to the server filesystem, XML parsing resolves external entities (XXE), and YAML deserialization uses `unsafe js-yaml.load`. **Controls:** None — unauthenticated access with no input validation.

Authenticated user chatbot session with external LLM

1. **[Crosses browser-to-server boundary]** The Angular SPA sends an HTTP POST to `/rest/chat` with a JSON body containing the messages array (chat history). The JWT Bearer token is attached in the Authorization header by the `RequestInterceptor`. No HTTPS enforcement exists. No rate limiting is applied. **Controls:** JWT is sent but the backend endpoint has no `express-jwt` authentication middleware.
2. **[Crosses request-handling to LLM prompt-construction boundary]** The Express route handler delegates to the `chat()` function in `routes/chat.ts`, carrying `req.body.messages` (user-controlled chat history) and request headers. The JWT is decoded but not cryptographically verified to resolve user identity. Untrusted user input meets the system prompt and tool definitions. **Controls:** None — no input sanitization or verification.
3. **[Crosses server-to-external-services boundary]** The chatbot sub-process sends an outbound HTTP request to the external LLM at `application.chatBot.llmApiUrl` using the `@ai-sdk/openai-compatible` provider. The payload includes the system prompt (containing confidential coupon policy), full user message history, and tool definitions. Authenticated via `LLM_API_KEY` environment variable. The LLM returns text deltas and tool-call instructions that the backend executes with no additional authorization checks. **Controls:** `LLM_API_KEY` authentication on outbound request; TLS assumed but not verified in source.

Profile image URL upload triggering server-side request forgery

1. **[Crosses browser-to-server boundary]** The Angular frontend sends an HTTP POST to `/profile/image/url` with a body containing the `imageUrl` field (an attacker-controlled URL string). The JWT Bearer token is sent in the Authorization header and a token cookie is included. No HTTPS enforcement exists. **Controls:** Cookie-based token lookup against an in-memory `authenticatedUsers` map; no `express-jwt` middleware guards this route.
2. **[Crosses server-to-external-services boundary]** The `profileImageUrlUpload` handler executes an outbound `fetch()` call to the arbitrary user-supplied URL using the backend server's network identity and `reach`. No URL validation, allowlist, or egress filtering is applied. The response body is streamed to the local filesystem. On fetch failure, the raw URL is stored in the database. **Controls:** None — unrestricted SSRF.

Real-time XSS/challenge verification via WebSocket

1. **[Crosses browser-to-server boundary]** The Angular SPA's `SocketIoService` establishes a `Socket.io` WebSocket connection to the backend. The browser emits events (`verifyLocalXssChallenge`, `verifySvgInjectionChallenge`, `verifyCloseNotificationsChallenge`) with string/array payloads containing XSS payload strings and SVG URLs. The server emits `challenge_solved` notification objects (including CTF flags) and `server_started` events to all connected clients. Transport is unencrypted HTTP. **Controls:** None — no authentication on WebSocket connection, no replay protection, CORS origin restriction ineffective against non-browser clients.
2. Socket event handlers invoke `challengeUtils.solveIf()` in-process, passing challenge references and criteria closures that evaluate user-supplied data from the WebSocket event payload. Challenge solved notifications including CTF flags are broadcast to all connected sockets.

File server access with directory traversal exploitation

1. **[Crosses browser-to-server boundary]** An unauthenticated user sends an HTTP GET request to `/ftp:/file`, `/ftp/quarantine:/file`, `/encryptionkeys:/file`, or `/support/logs:/file`. The

URL path contains the requested filename (user-controlled). No authentication is required. Transport is plaintext HTTP. **Controls:** None — unauthenticated access.

2. **[Crosses request-handling to filesystem boundary]** The file server calls `res.sendFile(path.resolve('ftp/', file))` or equivalent for other directories. The file parameter comes directly from `req.params.file` after only a forward-slash check and optional null-byte truncation (bypassable via poison null-byte injection). File contents are returned in the HTTP response body. **Controls:** Filename extension allowlist (bypassable); forward-slash check (bypassable via null-byte).

Product search and CRUD operations with SQL injection

1. **[Crosses browser-to-server boundary]** The Angular frontend sends HTTP GET/POST requests to `/rest/products/search?q=<user-input>`, `/api/Products`, `/api/BasketItems`, and `/api/Feedbacks`. Search queries are passed as URL query parameters; basket and feedback operations use JSON request bodies. The JWT Bearer token is attached by the `RequestInterceptor`, but the search endpoint requires no authentication. Transport is unencrypted HTTP. **Controls:** JWT authentication on `BasketItems` and `Feedbacks` endpoints; none on search.

2. **[Crosses application-to-database boundary]** For the search endpoint, `models.sequelize.query()` in `routes/search.ts` executes a raw SQL query with string-concatenated user input. No parameterization is applied. For CRUD endpoints, `finale-rest` uses Sequelize model methods with parameterized queries. Database credentials are hardcoded as `'username'/'password'`. **Controls:** Parameterized queries on CRUD endpoints only; none on search.

Initial page load serving compiled Angular frontend

1. Express static middleware reads compiled Angular bundle files from `frontend/dist/frontend/` directory via local filesystem access. No authentication is involved; files are pre-built at build time.

2. **[Crosses server-to-browser boundary]** The Express server delivers JavaScript bundles, HTML, CSS, and assets to the browser in HTTP responses. No authentication is required. The `X-Con-`

tent-Type-Options: nosniff header is set. No subresource integrity is enforced on served files.

Compression is enabled. Transport encryption depends on deployment configuration. **Controls:** nosniff header only; all client-side route guards, API endpoint definitions, and challenge hints are fully inspectable and modifiable by the end user.

Docker Compose container build and deployment with insecure configuration

1. **[Crosses operator-to-build-context boundary]** Docker build packages the entire repository root (context: `..`) into a container image via a multi-stage Dockerfile. The build context includes all source files, npm dependencies, and the `infrastructure/` directory containing a hardcoded TLS private key. No signature verification is applied to the resulting image. Transport is local Docker daemon IPC.

Controls: None — full source tree including secrets enters the build context.

Challenge solution triggering external webhook notification

1. **[Crosses browser-to-server boundary]** A user action (exploiting a vulnerability) triggers an HTTP request with a JWT Bearer token to a backend REST API endpoint. The specific payload varies by challenge type. **Controls:** JWT authentication where applicable to the triggering endpoint.

2. The route handler calls `challengeUtils.solveIf()` which invokes `solve()` and then `webhook.notify()` in-process. The webhook notifier sends an outbound HTTP POST to the URL specified in the `SOLUTIONS_WEBHOOK` environment variable. The payload contains challenge key, cheat score, hints data, CTF flag, and server metadata. **Controls:** None on the outbound call — no authentication headers, no certificate pinning, no TLS enforcement beyond URL scheme.

Security Posture

Authentication in OWASP Juice Shop is implemented via RS256 JWTs generated with a hardcoded private key embedded directly in source code (`lib/insecurity.ts`). The JWT contains user identity, role, and basket association with a 6-hour expiry. However, authentication middleware is applied inconsistently — critical endpoints including `/file-upload`, `/rest/chat`, product search, file serving, and WebSocket connections

require no authentication. The JWT private key's presence in source code means any reader of the repository can forge arbitrary tokens.

Encryption is effectively absent at the application layer. The server runs over plaintext HTTP by default (localhost:3000) with no TLS configuration in the source. Password storage uses unsalted MD5 hashing. The only encryption-related assets are RSA keys stored in the `encryptionkeys/` directory, which are served without authentication to any requester.

Network segmentation does not exist within the monolithic application. In the Docker Compose deployment, containers communicate over a bridge network with no authentication between services (Redis, MongoDB). CORS is configured to allow all origins (`app.use(cors())`). No rate limiting, WAF, or egress filtering is implemented. Access control relies entirely on inconsistently applied JWT middleware, with no defense-in-depth mechanisms.

Sensitive Assets

- **User credentials (email and password)** — Restricted. Submitted in plaintext HTTP POST body from frontend to backend. MD5-hashed (unsalted) for storage in SQLite. Extractable via SQL injection on the search endpoint.
- **JWT tokens** — Restricted. RS256-signed tokens containing user ID, email, role, and basket ID. Stored in browser `localStorage` and cookies (without `HttpOnly/Secure` flags). 6-hour expiry. Forgeable by anyone possessing the hardcoded private key.
- **JWT signing private key** — Restricted. RSA private key hardcoded as a string constant in `lib/insecurity.ts`. Possession allows arbitrary JWT forgery for any user or role. Not loaded from a secrets manager.
- **JWT public key (jwt.pub)** — Confidential. Stored in `encryptionkeys/jwt.pub` and served without authentication. Combined with the leaked private key, enables complete token forgery.
- **Premium key (premium.key)** — Confidential. Encryption key stored in `encryptionkeys/premium.key`, served without authentication.

- **LLM API credentials (LLM_API_KEY)** — Restricted. Environment variable used to authenticate outbound requests to the external LLM. Sent on every chatbot request.
- **CTF signing key (CTF_KEY)** — Restricted. HMAC key from environment variable or `ctf.key` file used to derive CTF flags. Compromise allows forging proof-of-solution.
- **CTF flags** — Confidential. HMAC-SHA1 of challenge names, broadcast to all connected WebSocket clients in challenge-solved notifications.
- **Hardcoded TLS private key** — Restricted. RSA private key committed in `infrastructure/terraform/networking.tf`. Used for AWS IAM server certificate. Exposed in source control.
- **System prompt with confidential policy** — Confidential. Contains bot identity, tool usage rules, and a confidential 15% courtesy discount escalation policy marked 'CONFIDENTIAL - INTERNAL ONLY'. Sent to external LLM on every chatbot request. Vulnerable to prompt injection extraction.
- **Coupon codes** — Confidential. Generated by `security.generateCoupon()` via z85-encoding of `month+discount`. Can be triggered via prompt injection against the LLM chatbot.
- **User chat messages** — Confidential. User-controlled message arrays passed unfiltered to the external LLM as conversation context. May contain personal information.
- **Uploaded file content** — Internal. Raw file bytes (ZIP/XML/YAML) held in memory after multer processing. Attacker-controlled content used to exploit XXE, zip-slip, and YAML deserialization.
- **Server filesystem data (XXE exfiltration targets)** — Restricted. Files readable by the Node.js process (e.g., `/etc/passwd`, application source, encryption keys) accessible via XML external entity resolution.
- **Internal network responses (SSRF targets)** — Confidential. HTTP responses from internal services/cloud metadata fetched via the SSRF vector and written to the public images directory.

- **SQLite database contents** — Restricted. Contains user credentials (email, MD5 password hashes), product data, basket items, feedback, and challenge state. Extractable via SQL injection including schema enumeration.
- **Application access logs** — Confidential. Files in `logs/` directory containing user IPs, request paths, timestamps. Served via `/support/logs/` without authentication.
- **Confidential documents (FTP directory)** — Confidential. Includes `acquisitions.md`, `package.json.bak`, `coupons_2013.md.bak`, `incident-support.kdbx`. Served with bypassable filename restrictions.
- **Frontend JavaScript bundle** — Internal. Compiled Angular application containing all client-side logic, API endpoint definitions, route guards, and challenge hints. Delivered to any unauthenticated user and fully inspectable.
- **Challenge solution data and webhook payloads** — Confidential. JSON containing challenge key, cheat score, CTF flag, and server metadata sent to external webhook without authentication.
- **Environment variables and Docker Compose configuration** — Confidential. Container environment (`NODE_ENV`, `REDIS_URL`), service topology, and port mappings. Contain intentional insecure configurations.
- **Backend source code (in container image)** — Internal. Complete source tree copied into Docker build context and persisted in image layers.
- **Search query parameter** — Internal. User-supplied string from `req.query.q` interpolated directly into SQL queries, serving as the injection vector.

Key Assumptions

- The backend serves the application over HTTP without TLS in default configuration; credentials and tokens travel in plaintext unless an external reverse proxy provides TLS termination.

- The cookie storing the JWT does not set HttpOnly or Secure flags, making the token accessible to client-side JavaScript via XSS.
- No rate limiting or account lockout mechanism exists on the `/rest/user/login` endpoint.
- The `/file-upload` endpoint has no server-side authentication enforcement — the route is registered without `security.isAuthenticated()` middleware.
- The `/rest/chat` endpoint has no express-jwt authentication middleware despite the frontend enforcing a login guard on the chatbot UI.
- The external LLM API endpoint communicates over HTTPS, protecting the `LLM_API_KEY` and prompt content in transit.
- The AI SDK executes tool calls returned by the LLM without additional authorization checks beyond what each tool's `execute` function implements.
- No network-level egress filtering restricts outbound HTTP from the backend process, making SSRF exploitable against internal services and cloud metadata endpoints.
- The Socket.IO WebSocket connection has no server-side authentication middleware; any client can connect and receive challenge-solved notifications including CTF flags.
- No WAF, reverse proxy path normalization, or network-level access control sits in front of the file-serving endpoints in typical deployment.
- The `SOLUTIONS_WEBHOOK` environment variable is set by an operator and not modifiable by application users at runtime.
- The `CTF_KEY` file or environment variable is provisioned securely and not exposed to end users.
- The Docker bridge network provides no encryption or authentication between containers on the same network.

- The hardcoded TLS private key in `networking.tf` is intentionally insecure as a CTF challenge and not representative of production deployment practices.

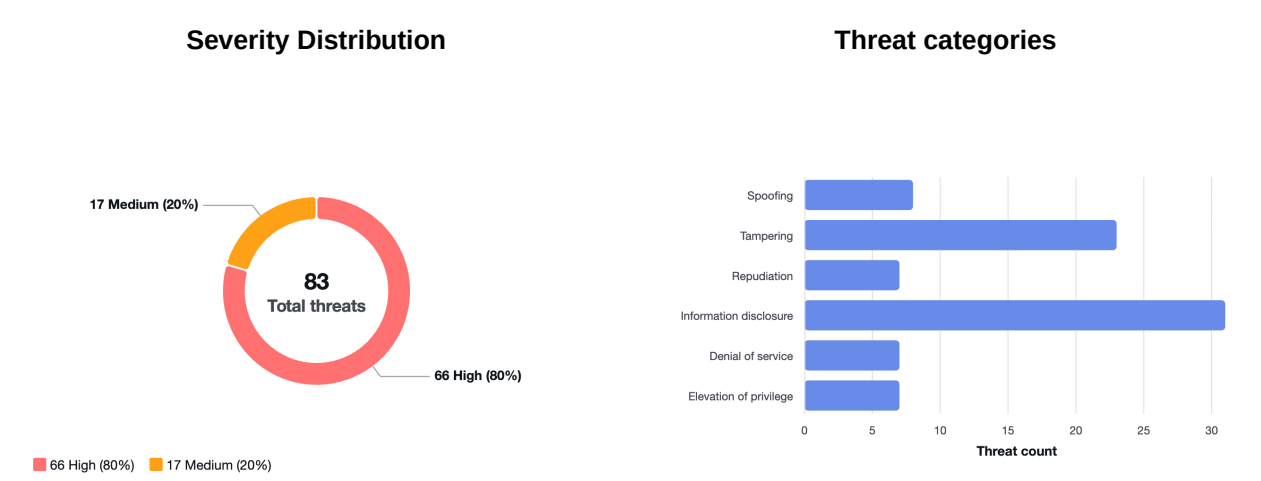
Configuration

Sources

Name	Type
juice-shop-master.zip	Object in S3 bucket juiceshop-ecs-securityagentre-sources-oz17q80ztcfr

Documents

Threats (83)



Threat #	Description	Severity	Status
1	The external webhook receiver operator, given that host-meta-data is sent in plaintext...	High	Open
2	A network attacker on the egress path, given that the outbound webhook call uses plain...	High	Open
3	An attacker who can modify docker-compose.yml can alter RE-DIS_URL or inject environment...	High	Open
4	An attacker with write access to the repository or build cache, given no image signing or...	High	Open
5	An attacker exploiting known vulnerabilities in MongoDB 4.4.29, given the intentionally...	High	Open
6	Any container on the juice-shop-net bridge network, given Mon-goDB runs without...	High	Open
7	Any container on the juice-shop-net bridge network, given Redis runs without...	High	Open
8	Any user with access to the container image, given the Docker build context includes the...	High	Open
9	An attacker with server filesystem access, gained through a separate backend...	High	Open
10	An end user can extract challenge hints and hacking-instructor tutorial steps from the...	High	Open

11	An end user can modify or disable client-side route guard logic in the delivered...	High	Open
12	An unauthenticated end user can inspect the delivered JavaScript bundle to enumerate all...	High	Open
13	A network attacker (on-path), when TLS is not enforced at deployment, can intercept and...	High	Open
14	An anonymous end user performing SQL injection attacks against /rest/products/search...	High	Open
15	A network-adjacent attacker positioned on the same network segment can passively sniff...	High	Open
16	An authenticated end user holding any valid JWT can submit POST /api/Feedbacks with a...	High	Open
17	Any user with source code access who obtains the hardcoded RSA private key from...	High	Open
18	An anonymous end user can send PUT requests to /api/Products/:id to modify product name,...	High	Open
19	An anonymous end user can inject a UNION SELECT against sqlite_master through the search...	High	Open
20	An anonymous end user can craft a UNION SELECT payload in the search query parameter to...	High	Open
21	An anonymous end user can inject arbitrary SQL statements through the q parameter in...	High	Open
22	Attacker who achieved directory traversal write or Node.js process compromise with...	High	Open
23	End User (Browser) can browse serve-index directory listings at /ftp, /encryptionkeys,...	High	Open
24	End User (Browser) can access sensitive files via file-serving endpoints without any...	High	Open
25	Network-positioned attacker who can intercept network traffic between user and server due...	High	Open
26	End User (Browser) can request /support/logs/ to enumerate and download application...	High	Open
27	End User (Browser) who knows that forward-slash check can be bypassed with URL-encoded or...	High	Open
28	End User (Browser) can request /encryptionkeys/premium.key without authentication to...	High	Open

29	End User (Browser) can request /encryptionkeys/jwt.pub directly without authentication to...	High	Open
30	End User (Browser) who knows the null-byte injection technique (%2500) and the file...	High	Open
31	An unauthenticated WebSocket client who knows expected payload patterns can trigger...	High	Open
32	A network-level attacker in a shared lab environment can intercept unencrypted WebSocket...	High	Open
33	A late-connecting WebSocket client can receive the full replay of cached notifications...	High	Open
34	Any connected WebSocket client can passively receive broadcast 'challenge solved'...	High	Open
35	A malicious participant who can connect to the unauthenticated WebSocket endpoint can...	High	Open
36	An external attacker with network access to port 3000 can connect to the Socket.IO...	High	Open
37	An authenticated end-user can store an attacker-controlled URL in the profileImage...	High	Open
38	An authenticated end-user can perform SSRF requests through the profile image URL handler...	High	Open
39	A network-adjacent attacker on the same LAN can sniff unencrypted HTTP traffic carrying...	High	Open
40	An authenticated end-user controlling an external server can serve arbitrary malicious...	High	Open
41	An external attacker with a stolen cookie token can invoke /profile/image/url without a...	High	Open
42	An authenticated end-user can supply a malicious URL that fails to fetch, causing the raw...	High	Open
43	An authenticated end-user with a valid session cookie can supply a URL targeting internal...	High	Open
44	An end user, given that user messages are unfiltered and JWT identity is decoded without...	High	Open
45	An anonymous internet attacker, given that no rate limiting or authentication is applied...	High	Open
46	The external LLM API, given that no TLS enforcement is verified in source and the URL...	High	Open

47	An anonymous internet attacker, given that security.decode uses jws.decode without...	High	Open
48	An end user, given that the getProductReviews tool uses MongoDB \$where with string...	High	Open
49	An end user can craft chat messages that instruct the LLM to reveal its system prompt...	High	Open
50	An end user can craft adversarial chat messages that override the system prompt...	High	Open
51	An anonymous internet attacker, given that the /rest/chat endpoint has no...	High	Open
52	An anonymous internet attacker, given that no authentication is enforced and no user...	High	Open
53	A network-positioned attacker, given that the application defaults to HTTP without TLS...	High	Open
54	An anonymous internet attacker, given that parsed XML with resolved entities is returned...	High	Open
55	An anonymous internet attacker, given that checkFileType and checkUploadSize only log...	High	Open
56	An anonymous internet attacker, given that YAML parsing uses js-yaml.load (not safeLoad)...	High	Open
57	An anonymous internet attacker, given that ZIP extraction uses an insufficient...	High	Open
58	An anonymous internet attacker, given that the XML parser is configured with...	High	Open
59	An anonymous internet attacker, given that the /file-upload endpoint has no server-side...	High	Open
60	An attacker with database read access via SQL injection can perform offline rainbow-table...	High	Open
61	A network-level attacker in MITM position, given application uses HTTP without TLS, can...	High	Open
62	An external attacker, given no rate limiting or account lockout exists on...	High	Open
63	An attacker exploiting XSS in the Juice Shop frontend, given the token is stored in...	High	Open
64	An external attacker with access to public source code, given the RSA private key is...	High	Open

65	A network-level attacker, given application deployed with default HTTP configuration...	High	Open
66	An end-user-via-browser can inject SQL payload into the email field which is interpolated...	High	Open
67	An unresponsive external webhook receiver can cause the out-bound fetch in lib/webhook.ts...	Medium	Open
68	An external attacker who knows the webhook URL can send crafted POST requests to the...	Medium	Open
69	An attacker with environment variable write access, given that webhook-url has no...	Medium	Open
70	An operator with Docker socket access can build and deploy a tampered container image and...	Medium	Open
71	A malicious third-party website, exploiting the fully open CORS policy (app.options('*',...)	Medium	Open
72	An anonymous end user can inject computationally expensive SQL statements through the...	Medium	Open
73	An authenticated end user holding any valid JWT can send POST/PUT requests to...	Medium	Open
74	An external attacker can open thousands of concurrent unauthenticated WebSocket...	Medium	Open
75	A malicious training participant can deny triggering challenge solves since no identity...	Medium	Open
76	A malicious WebSocket client can send malformed or oversized payloads through...	Medium	Open
77	An authenticated end-user can craft a URL with a manipulated extension or path component,...	Medium	Open
78	An authenticated end-user can supply URLs pointing to extremely large or slow-streaming...	Medium	Open
79	The external LLM API, given that full user message history is forwarded on every request,...	Medium	Open
80	The external LLM API, given that the AI SDK executes tool calls without additional...	Medium	Open
81	An anonymous internet attacker, given that ZIP extraction writes to the server filesystem...	Medium	Open
82	An authenticated end user can modify the basket ID in session-Storage or API requests to...	Medium	Open

83

An end user or attacker using compromised credentials can perform account actions and...

Medium

Open

The external webhook receiver operator, given that host-metadata is sent in plaintext with no...

Severity:	High
Status:	Open
STRIDE:	Information disclosure
Affected Assets:	host-metadata
Identified:	August 13, 2026

Threat Source

External Webhook Receiver operator

Prerequisites

SOLUTIONS_WEBHOOK is configured to point to a third-party service whose data handling policies are unknown or untrusted

Threat Action

Collect and retain server hostname, OS type/release, NODE_ENV, and application version from every webhook notification, building a detailed fingerprint of the deployment environment

Impact

Exposure of infrastructure details (hostname, OS release, Node environment) to an external party, enabling targeted exploitation of known OS or application version vulnerabilities

Evidence

- lib/webhook.ts lines 29-35 include hostname, OS type/release, app name, NODE_ENV, and version in the issuer block of every webhook POST payload.

Anchor

node: host-metadata

A network attacker on the egress path, given that the outbound webhook call uses plain HTTP with no...

Severity:	High
Status:	Open
STRIDE:	Information disclosure
Affected Assets:	ctf-flag
Identified:	August 13, 2026

Threat Source

Network attacker on egress path

Prerequisites

Attacker has ability to intercept traffic between backend and external webhook receiver (e.g., shared network, compromised DNS, or SOLUTIONS_WEBHOOK set to HTTP scheme)

Threat Action

Intercept the plaintext JSON POST containing ctfFlag HMAC values, challenge solution data, and host-metadata (hostname, OS type, app version) since no TLS enforcement or certificate pinning is coded in lib/webhook.ts

Impact

Disclosure of CTF competition flags allowing an attacker to submit them without solving challenges, plus infrastructure fingerprinting data enabling further targeted attacks

Evidence

- lib/webhook.ts sends the payload via fetch with Content-Type application/json and no TLS enforcement; posture confirms no authentication or signing on outbound request.

Anchor

flow: Backend REST API ' Challenge Detection & Webhook Notifier

An attacker who can modify docker-compose.yml can alter REDIS_URL or inject environment overrides...

Severity:	High
Status:	Open
STRIDE:	Tampering
Affected Assets:	environment-variables
Identified:	August 13, 2026

Threat Source

Attacker who can modify docker-compose.yml or inject environment overrides

Prerequisites

Attacker has write access to docker-compose.yml or can supply a docker-compose.override.yml in the same directory; operator does not review environment changes before deployment

Threat Action

Modify REDIS_URL or add environment variables to redirect the backend container to an attacker-controlled Redis or MongoDB instance, intercepting all session and application data

Impact

Backend connects to attacker-controlled data services, enabling full credential interception, session manipulation, and data exfiltration

Evidence

- docker-compose.yml defines REDIS_URL=redis://redis:6379 without authentication per code_sc-b5c04c05-c24b-446e-ae94-9fbcea71dc7a/juice-shop-master/infrastructure/docker-compose.yml.

Anchor

node: Infrastructure Build Orchestration

An attacker with write access to the repository or build cache, given no image signing or...

Severity:	High
Status:	Open
STRIDE:	Tampering
Affected Assets:	container-image
Identified:	August 13, 2026

Threat Source

Attacker with write access to the repository or Docker build cache

Prerequisites

Attacker can modify files in the repository root (which serves as Docker build context) or poison the npm registry responses during build; no image signing or verification is configured

Threat Action

Inject malicious code into the source tree or supply-chain dependencies before or during the Docker build, producing a tampered container image that appears legitimate

Impact

A compromised container image runs arbitrary attacker code with the backend's privileges, enabling data theft, backdoor persistence, and lateral movement

Evidence

- The Dockerfile copies the entire repository root and runs npm install without integrity verification; no image signing is configured per `code_sc-b5c04c05-c24b-446e-ae94-9fbeea71dc7a/juice-shop-master/`

Anchor

node: container-image

An attacker exploiting known vulnerabilities in MongoDB 4.4.29, given the intentionally outdated...

Severity:	High
Status:	Open
STRIDE:	Denial of service
Affected Assets:	container-image
Identified:	August 13, 2026

Threat Source

Attacker exploiting known vulnerabilities in MongoDB 4.4.29

Prerequisites

MongoDB 4.4.29 is deployed (intentionally outdated for vulnerableDockerImageChallenge); attacker has network access to the container via the bridge network

Threat Action

Exploit known CVEs in the outdated MongoDB 4.4.29 image to crash the database, cause denial of service, or achieve remote code execution within the MongoDB container

Impact

Database unavailability, data corruption, or container escape leading to broader infrastructure compromise

Evidence

- docker-compose.yml explicitly uses mongo:4.4.29 which is marked as vulnerableDockerImageChallenge per code_sc-b5c04c05-c24b-446e-ae94-9fbeea71dc7a/juice-shop-master/infrastructure/docker-compose.yml.

Anchor

node: MongoDB Store

Any container on the juice-shop-net bridge network, given MongoDB runs without authentication, can...

Severity:	High
Status:	Open
STRIDE:	Information disclosure
Affected Assets:	docker-compose-configuration
Identified:	August 13, 2026

Threat Source

Any container on the juice-shop-net bridge network

Prerequisites

Attacker gains code execution in any container on the shared Docker bridge network; MongoDB 4.4.29 runs without authentication

Threat Action

Connect to MongoDB port 27017 without credentials and dump all collections containing application data including user records, orders, and challenge progress

Impact

Full disclosure of application data stored in MongoDB, including any user-submitted information, product data, and internal challenge state

Evidence

- docker-compose.yml deploys MongoDB 4.4.29 without authentication and it is accessible on the internal juice-shop-net network per code_sc-b5c04c05-c24b-446e-ae94-9fbeea71dc7a/juice-shop-master/infrastr

Anchor

node: MongoDB Store

Any container on the juice-shop-net bridge network, given Redis runs without authentication, can...

Severity:	High
Status:	Open
STRIDE:	Tampering
Affected Assets:	environment-variables
Identified:	August 13, 2026

Threat Source

Any container on the juice-shop-net bridge network

Prerequisites

Attacker has code execution in any container on the juice-shop-net Docker bridge network (e.g., via an application vulnerability in the backend); Redis runs without requirepass

Threat Action

Connect to Redis on port 6379 without credentials and modify or overwrite session/cache data, inject arbitrary keys, or flush the store

Impact

Session hijacking, privilege escalation by modifying session attributes, or cache poisoning that alters application behavior for other users

Evidence

- docker-compose.yml configures Redis 7.4.4-alpine without requirepass and REDIS_URL=redis://redis:6379 contains no credentials per code_sc-b5c04c05-c24b-446e-ae94-9fbeea71dc7a/juice-shop-master/infrast

Anchor

node: Redis Cache

Any user with access to the container image, given the Docker build context includes the full...

Severity:	High
Status:	Open
STRIDE:	Information disclosure
Affected Assets:	hardcoded-tls-private-key
Identified:	August 13, 2026

Threat Source

Any user with access to the container image

Prerequisites

Docker build context set to repository root (context: ../), which includes infrastructure/terraform/networking.tf containing the hardcoded RSA private key

Threat Action

Extract the TLS private key from intermediate or final image layers by inspecting the Docker image history or layer filesystem, since the entire repository (including networking.tf with iacLeakedKeyChallenge) is available in the build context

Impact

Disclosure of RSA private key enables TLS impersonation, man-in-the-middle attacks against the ALB HTTPS listener, and decryption of intercepted traffic

Evidence

- infrastructure/Dockerfile uses 'context: ../' per docker-compose.yml, and networking.tf contains a hardcoded RSA private key marked as iacLeakedKeyChallenge per code_sc-b5c04c05-c24b-446e-ae94-9fbeea7

Anchor

flow: Infrastructure Build Orchestration -> Backend Container Image

An attacker with server filesystem access, gained through a separate backend vulnerability such as...

Severity:	High
Status:	Open
STRIDE:	Tampering
Affected Assets:	Frontend JavaScript Bundle
Identified:	August 13, 2026

Threat Source

attacker with server filesystem access

Prerequisites

Attacker gains write access to the server filesystem through a separate backend vulnerability (e.g., file upload path traversal in routes/fileUpload.ts)

Threat Action

Modify JavaScript files in frontend/dist/frontend/ to inject persistent malicious client-side code that is served to all subsequent users

Impact

All users loading the application receive trojaned JavaScript, enabling mass credential theft, session hijacking, or client-side crypto-mining

Evidence

- server.ts serves static files from 'frontend/dist/frontend' (line ~291); routes/fileUpload.ts is listed as a key backend file with known vulnerability patterns; no subresource integrity is enforced on

Anchor

node: Frontend Dist Directory

An end user can extract challenge hints and hacking-instructor tutorial steps from the compiled...

Severity:	High
Status:	Open
STRIDE:	Information disclosure
Affected Assets:	Challenge Hints
Identified:	August 13, 2026

Threat Source

end-user-via-browser

Prerequisites

#Empty

Threat Action

Extract hacking-instructor tutorial steps and challenge hints from the compiled frontend bundle without solving challenges, revealing intended exploitation paths

Impact

CTF participants or trainees bypass the learning process by reading solutions directly from the bundle, undermining the pedagogical value and competitive integrity of challenges

Evidence

- Posture identifies 'hacking-instructor module compiled into the frontend bundle' with 'Confidentiality matters — reveals intended vulnerability exploitation paths.' Origin is Frontend Dist Directory.

Anchor

node: Challenge Hints

An end user can modify or disable client-side route guard logic in the delivered JavaScript bundle,...

Severity:	High
Status:	Open
STRIDE:	Tampering
Affected Assets:	Client-Side Route Guards
Identified:	August 13, 2026

Threat Source

end-user-via-browser

Prerequisites

#Empty

Threat Action

Modify or disable the delivered LoginGuard, AdminGuard, AccountingGuard, and DeluxeGuard logic in browser DevTools or via a proxy, gaining client-side access to protected routes without valid authorization

Impact

User accesses admin or restricted UI views client-side, which may expose sensitive UI elements or enable crafting of privileged API calls discovered through those views

Evidence

- app.guard.ts defines LoginGuard, AdminGuard, AccountingGuard, and DeluxeGuard that decode JWT client-side; posture states 'these are advisory-only controls that can be bypassed by modifying the delive

Anchor

node: Client-Side Route Guards

An unauthenticated end user can inspect the delivered JavaScript bundle to enumerate all backend...

Severity:	High
Status:	Open
STRIDE:	Information disclosure
Affected Assets:	API Endpoint Definitions
Identified:	August 13, 2026

Threat Source

end-user-via-browser (unauthenticated)

Prerequisites

#Empty

Threat Action

Inspect the delivered JavaScript bundle to enumerate all backend REST API paths, parameters, and hidden administrative endpoints embedded in Angular service files

Impact

Attacker gains a complete map of the application's attack surface, including undocumented or admin-only endpoints, enabling targeted exploitation of backend vulnerabilities

Evidence

- Posture notes API endpoint definitions are 'embedded in Angular service files compiled into the bundle' and 'Confidentiality matters — exposes the full attack surface to any user.' Key files include f

Anchor

node: API Endpoint Definitions

A network attacker (on-path), when TLS is not enforced at deployment, can intercept and modify the...

Severity:	High
Status:	Open
STRIDE:	Tampering
Affected Assets:	Frontend JavaScript Bundle
Identified:	August 13, 2026

Threat Source

network attacker (on-path)

Prerequisites

TLS not enforced at deployment — server.ts does not configure TLS directly, relying on external infrastructure per assumptions

Threat Action

Intercept and modify the HTTP response carrying JavaScript bundles between server and browser, injecting malicious code (e.g., credential-harvesting script)

Impact

User executes attacker-controlled JavaScript, leading to credential theft, session hijacking, or arbitrary actions performed as the victim

Evidence

- server.ts does not configure TLS (assumption noted in posture); hop 2 states 'Transport encryption depends on deployment configuration (not enforced in source)' and 'No subresource integrity enforced'

Anchor

flow: Express Static File Server ' End User Browser

An anonymous end user performing SQL injection attacks against /rest/products/search without any...

Severity:	High
Status:	Open
STRIDE:	Repudiation
Affected Assets:	User Credentials in Database
Identified:	August 13, 2026

Threat Source

anonymous end user

Prerequisites

the search endpoint requires no authentication, so requests carry no identity binding

Threat Action

perform SQL injection attacks against /rest/products/search without any authenticated session, leaving no user-attributable audit trail

Impact

inability to attribute data exfiltration or tampering attacks to a specific actor, hindering incident response and forensics

Evidence

- The posture confirms /rest/products/search requires no authentication; no logging/audit mechanism is documented for this endpoint in routes/search.ts or server.ts.

Anchor

node: End User

A network-adjacent attacker positioned on the same network segment can passively sniff unencrypted...

Severity:	High
Status:	Open
STRIDE:	Information disclosure
Affected Assets:	JWT Bearer Token
Identified:	August 13, 2026

Threat Source

network-adjacent attacker

Prerequisites

attacker is positioned on the same network segment as the user (e.g., shared WiFi), which is common in lab/training environments where Juice Shop is deployed

Threat Action

passively sniff unencrypted HTTP traffic between the browser and the backend to capture JWT Bearer tokens and request/response payloads

Impact

stolen JWT tokens allow full session hijacking; captured responses may contain sensitive product, user, or feedback data

Evidence

- The posture assumption states 'the application is served over unencrypted HTTP in typical deployments (localhost:3000 default), as no TLS configuration is present in source'.

Anchor

flow: Angular Frontend SPA -> REST API Sub-process

An authenticated end user holding any valid JWT can submit POST /api/Feedbacks with a manipulated...

Severity:	High
Status:	Open
STRIDE:	Tampering
Affected Assets:	User Feedback
Identified:	August 13, 2026

Threat Source

authenticated end user

Prerequisites

attacker holds any valid JWT (obtainable by registering a free account or forging one with the hardcoded key)

Threat Action

submit POST /api/Feedbacks with a manipulated UserId field in the JSON body to attribute feedback to a different user

Impact

forged feedback attributed to other users, enabling reputation manipulation and social engineering

Evidence

- The posture notes 'forged feedback challenge shows UserId can be tampered'; finale-rest CRUD does not validate UserId matches the token subject.

Anchor

node: User Feedback

Any user with source code access who obtains the hard-coded RSA private key from lib/insecurity.ts...

Severity:	High
Status:	Open
STRIDE:	Information disclosure
Affected Assets:	JWT Bearer Token
Identified:	August 13, 2026

Threat Source

any user with source code access

Prerequisites

attacker obtains the hardcoded RSA private key from the open-source repository (lib/insecurity.ts lines 24-26)

Threat Action

sign arbitrary JWT tokens for any user identity or admin role using the exposed private key

Impact

full authentication bypass enabling impersonation of any user including administrator accounts

Evidence

- lib/insecurity.ts lines 24-26 contain the hardcoded RSA private key; the posture notes 'anyone with source access can forge valid JWTs for any user/role'.

Anchor

node: JWT Private Key

An anonymous end user can send PUT requests to /api/Products/:id to modify product name,...

Severity:	High
Status:	Open
STRIDE:	Elevation of privilege
Affected Assets:	Product Data
Identified:	August 13, 2026

Threat Source

anonymous end user

Prerequisites

#Empty

Threat Action

send PUT requests to /api/Products/:id to modify product name, description, or price, exploiting the commented-out authorization middleware (server.ts ~line 373)

Impact

arbitrary product data manipulation including price alteration and injection of malicious content into product descriptions

Evidence

- The posture's distinctive section notes 'The PUT /api/Products/:id endpoint has its authorization middleware commented out (server.ts ~line 373), enabling unauthenticated product modification'.

Anchor

node: REST API Sub-process

An anonymous end user can inject a UNION SELECT against sqlite_master through the search endpoint...

Severity:	High
Status:	Open
STRIDE:	Information disclosure
Affected Assets:	SQLite Database Schema
Identified:	August 13, 2026

Threat Source

anonymous end user

Prerequisites

#Empty

Threat Action

inject a UNION SELECT against sqlite_master through the search endpoint to enumerate all table and column definitions

Impact

full database schema exposed, enabling targeted exploitation of other tables and sensitive columns

Evidence

- The posture notes SQLite Database Schema is derived from Search Query Parameter via sql-injection-extract; routes/search.ts confirms the dbSchemaChallenge validates this extraction.

Anchor

node: SQLite Database Schema

An anonymous end user can craft a UNION SELECT payload in the search query parameter to extract...

Severity:	High
Status:	Open
STRIDE:	Information disclosure
Affected Assets:	User Credentials in Database
Identified:	August 13, 2026

Threat Source

anonymous end user

Prerequisites

#Empty

Threat Action

craft a UNION SELECT payload in the search query parameter to extract email addresses and MD5 password hashes from the Users table via the unauthenticated /rest/products/search endpoint

Impact

exposure of all user credentials enabling offline password cracking and account takeover

Evidence

- routes/search.ts concatenates user input into raw SQL; the posture notes User Credentials are 'extractable via UNION-based SQL injection on the search endpoint'.

Anchor

node: User Credentials in Database

An anonymous end user can inject arbitrary SQL statements through the q parameter in...

Severity:	High
Status:	Open
STRIDE:	Tampering
Affected Assets:	Product Data
Identified:	August 13, 2026

Threat Source

anonymous end user

Prerequisites

#Empty

Threat Action

inject arbitrary SQL statements through the q parameter in /rest/products/search, which is concatenated directly into a raw SQL query string (routes/search.ts line 23) without parameterization

Impact

modification of product records, deletion of database tables, or insertion of malicious data into any accessible table

Evidence

- routes/search.ts line 23 shows direct string interpolation: `SELECT \* FROM Products WHERE ((name LIKE '%\${criteria}%') ...)`.

Anchor

flow: REST API Sub-process -> SQLite Database

Attacker who achieved directory traversal write or Node.js process compromise with file-write...

Severity:	High
Status:	Open
STRIDE:	Tampering
Affected Assets:	JWT Public Key (jwt.pub)
Identified:	August 13, 2026

Threat Source

Attacker who achieved directory traversal write or Node.js process compromise

Prerequisites

Attacker achieves code execution or file-write capability on the server (e.g., via file upload vulnerability in another trace combined with path traversal)

Threat Action

Overwrite jwt.pub or premium.key in the encryptionkeys/ directory to substitute attacker-controlled keys, enabling authentication bypass or content manipulation

Impact

JWT verification uses attacker's public key allowing forged tokens to validate, or premium content encryption is compromised

Evidence

- Posture confirms encryptionkeys/ directory served without auth and keys used by express-jwt for verification; lib/insecurity.ts references these key files for JWT operations.

Anchor

node: Encryption Keys Directory

End User (Browser) can browse serve-index directory listings at /ftp, /encryptionkeys, and...

Severity:	High
Status:	Open
STRIDE:	Information disclosure
Affected Assets:	Confidential Documents (FTP)
Identified:	August 13, 2026

Threat Source

End User (Browser)

Prerequisites

#Empty

Threat Action

Browse serve-index directory listings at /ftp, /encryptionkeys, and /support/logs to enumerate all available sensitive files and plan targeted exfiltration

Impact

Complete knowledge of available attack surface including filenames, file sizes, and directory structure that accelerates exploitation of other vulnerabilities

Evidence

- Posture assumption confirms serve-index middleware exposes full directory listings without authentication; server.ts registers serve-index at these paths.

Anchor

node: Backend Express Server

End User (Browser) can access sensitive files via file-serving endpoints without any...

Severity:	High
Status:	Open
STRIDE:	Repudiation
Affected Assets:	Confidential Documents (FTP)
Identified:	August 13, 2026

Threat Source

End User (Browser)

Prerequisites

#Empty

Threat Action

Access sensitive files (/encryptionkeys/, /ftp/, /support/logs/) without any authentication, making it impossible to attribute file downloads to a specific user identity

Impact

No forensic trail connecting sensitive file exfiltration to a specific actor; inability to detect or investigate data theft

Evidence

- Posture trust summary confirms no authentication middleware on file-serving routes; server.ts lines 276-289 register serve-index without auth middleware.

Anchor

node: End User (Browser)

Network-positioned attacker who can intercept network traffic between user and server due to...

Severity:	High
Status:	Open
STRIDE:	Information disclosure
Affected Assets:	JWT Public Key (jwt.pub)
Identified:	August 13, 2026

Threat Source

Network-positioned attacker (man-in-the-middle)

Prerequisites

Attacker can intercept network traffic between user and server (e.g., shared WiFi, compromised router) since no TLS enforcement is configured in server.ts

Threat Action

Passively capture HTTP responses containing encryption keys, confidential documents, or log files transmitted in plaintext between the backend and browser

Impact

Exposure of JWT public key, premium key, confidential business documents, and user log data to network observers

Evidence

- Posture assumption states the application is served over plaintext HTTP with no TLS configuration observed in server.ts.

Anchor

flow: End User (Browser) ' Backend Express Server

End User (Browser) can request /support/logs/ to enumerate and download application access log...

Severity:	High
Status:	Open
STRIDE:	Information disclosure
Affected Assets:	Log Files
Identified:	August 13, 2026

Threat Source

End User (Browser)

Prerequisites

#Empty

Threat Action

Request /support/logs/ to enumerate and download application access log files containing user IP addresses, request paths, timestamps, and potentially session identifiers

Impact

Disclosure of user PII (IP addresses), application usage patterns, and potentially session tokens or authentication cookies logged in request data

Evidence

- Posture confirms logs served via /support/logs/:file without authentication; server.ts uses file-stream-rotator to write access logs to logs/ directory.

Anchor

node: Log Files

End User (Browser) who knows that forward-slash check can be bypassed with URL-encoded or...

Severity:	High
Status:	Open
STRIDE:	Tampering
Affected Assets:	Server Filesystem Paths
Identified:	August 13, 2026

Threat Source

End User (Browser)

Prerequisites

Attacker knows that forward-slash check can be bypassed with URL-encoded or alternative path separator sequences

Threat Action

Craft a filename parameter with encoded directory traversal sequences (e.g., `..%5c` or `..%252f`) that bypass the forward-slash-only check but are resolved by `path.resolve()` to access files outside the intended directory

Impact

Read arbitrary files from the server filesystem accessible to the Node.js process, including application source code, configuration files, and OS credentials

Evidence

- Posture confirms `path.resolve()` is used with user-controlled filename param and the only guard is a forward-slash check; `routes/fileServer.ts` and other route handlers documented in posture.

Anchor

flow: File Server Sub-process ' FTP Directory

End User (Browser) can request /encryptionkeys/premium.key without authentication to download the...

Severity:	High
Status:	Open
STRIDE:	Information disclosure
Affected Assets:	Premium Key (premium.key)
Identified:	August 13, 2026

Threat Source

End User (Browser)

Prerequisites

#Empty

Threat Action

Request /encryptionkeys/premium.key without authentication to download the premium content encryption key

Impact

Attacker obtains the encryption key protecting premium content, enabling decryption or unauthorized access to premium features

Evidence

- Posture confirms premium.key is served without authentication from routes/keyServer.ts; server.ts registers /encryptionkeys with serve-index for directory browsing.

Anchor

node: Premium Key (premium.key)

End User (Browser) can request /encryption-keys/jwt.pub directly without authentication to obtain...

Severity:	High
Status:	Open
STRIDE:	Information disclosure
Affected Assets:	JWT Public Key (jwt.pub)
Identified:	August 13, 2026

Threat Source

End User (Browser)

Prerequisites

#Empty

Threat Action

Request /encryptionkeys/jwt.pub directly without authentication to obtain the RSA public key, which combined with the hardcoded private key in lib/insecurity.ts enables JWT token forgery

Impact

Attacker obtains the JWT verification key and can forge arbitrary JWT tokens to impersonate any user including admin accounts

Evidence

- Posture confirms encryptionkeys/jwt.pub is served without authentication via routes/keyServer.ts; lib/insecurity.ts contains the hardcoded RSA private key enabling full key-pair compromise.

Anchor

node: JWT Public Key (jwt.pub)

End User (Browser) who knows the null-byte injection technique (%2500) and the file naming...

Severity:	High
Status:	Open
STRIDE:	Elevation of privilege
Affected Assets:	Confidential Documents (FTP)
Identified:	August 13, 2026

Threat Source

End User (Browser)

Prerequisites

Attacker knows the null-byte injection technique (%2500) and the file naming convention in /ftp directory

Threat Action

Inject a poison null-byte sequence into the filename parameter (e.g., /ftp/package.json.bak%2500.md) to bypass the .md/.pdf extension allowlist in servePublicFiles handler

Impact

Access to files with disallowed extensions (.bak, .yaml, .kdbx, .pyc) containing confidential business data, coupon codes, and credential databases

Evidence

- routes/fileServer.ts uses cutOffPoisonNullByte before extension check, confirmed in posture as an explicit challenge design; server.ts registers /ftp route with serve-index.

Anchor

node: File Server Sub-process

An unauthenticated WebSocket client who knows expected payload patterns can trigger challenge...

Severity:	High
Status:	Open
STRIDE:	Tampering
Affected Assets:	Challenge Solved State
Identified:	August 13, 2026

Threat Source

Unauthenticated WebSocket client

Prerequisites

Attacker can connect to Socket.IO endpoint and knows the expected payload pattern for a challenge

Threat Action

Trigger `challengeUtils.solveSelf()` by sending matching payloads over the WebSocket, causing `challenge.save()` to persist `solved=true` in SQLite without any authorization check at the persistence boundary

Impact

Challenge records are permanently marked as solved, corrupting the training state for the instance; other users see challenges pre-solved

Evidence

- Posture describes trust boundary 'WebSocket Handler to Persistence' with no additional authorization; `challenge.save()` is called directly after criteria match in `lib/challengeUtils.ts`.

Anchor

node: Challenges Store (SQLite)

A network-level attacker in a shared lab environment can intercept unencrypted WebSocket traffic,...

Severity:	High
Status:	Open
STRIDE:	Information disclosure
Affected Assets:	CTF Flags
Identified:	August 13, 2026

Threat Source

Network-level attacker (same LAN, shared lab environment)

Prerequisites

Application runs over unencrypted HTTP on localhost:3000 (default deployment); attacker has network proximity to sniff traffic

Threat Action

Intercept WebSocket frames via packet capture to extract XSS verification payloads and challenge-solved notification objects containing CTF flags

Impact

CTF flags and challenge solutions are disclosed to a passive network observer

Evidence

- Posture assumption states the application runs over unencrypted HTTP (localhost:3000) and WebSocket traffic is not TLS-protected.

Anchor

flow: SocketIoService (Frontend) -> WebSocket Events Handler (Backend)

A late-connecting WebSocket client can receive the full replay of cached notifications including...

Severity:	High
Status:	Open
STRIDE:	Information disclosure
Affected Assets:	Challenge Solved Notifications
Identified:	August 13, 2026

Threat Source

Late-connecting WebSocket client

Prerequisites

At least one challenge has been solved previously; attacker connects after the fact (no authentication required)

Threat Action

Connect to the Socket.IO endpoint and receive the full replay of all stored notification objects from the in-memory notifications array, which includes historical CTF flags

Impact

Complete set of previously earned CTF flags is disclosed to the late-connecting attacker in one batch

Evidence

- Posture describes notifications[] in data/datacache.ts being replayed to every newly connecting socket; no auth gates this replay.

Anchor

node: Notifications Cache

Any connected WebSocket client can passively receive broadcast 'challenge solved' notifications...

Severity:	High
Status:	Open
STRIDE:	Information disclosure
Affected Assets:	CTF Flags
Identified:	August 13, 2026

Threat Source

Any connected WebSocket client (passive eavesdropper)

Prerequisites

Attacker has an open Socket.IO connection to the backend (no authentication required)

Threat Action

Passively listen on the WebSocket for 'challenge solved' events that are broadcast to ALL connected sockets, capturing CTF flags intended for the solving participant

Impact

CTF flags for all solved challenges are disclosed to unauthorized participants, breaking CTF competition fairness

Evidence

- Posture confirms notifications are broadcast to all sockets; challengeUtils.ts constructs notification objects with ctfFlag() and broadcasts via global io instance.

Anchor

node: CTF Flags

A malicious participant who can connect to the unauthenticated WebSocket endpoint can emit crafted...

Severity:	High
Status:	Open
STRIDE:	Tampering
Affected Assets:	Challenge Solved State
Identified:	August 13, 2026

Threat Source

Malicious training participant or external attacker

Prerequisites

Attacker can connect to the WebSocket endpoint (no authentication required, reachable on port 3000)

Threat Action

Emit a 'verifyLocalXssChallenge' event with the expected XSS pattern string directly over the WebSocket without actually triggering the XSS in the DOM

Impact

Challenge is marked as solved in SQLite and CTF flag is generated and broadcast, undermining the training exercise integrity

Evidence

- WebSocket Events Handler checks payloads against expected patterns (registerWebsocketEvents.ts) but has no proof-of-work or DOM context verification; posture marks XSS Verification Payloads origin as

Anchor

flow: SocketIoService (Frontend) -> WebSocket Events Handler (Backend)

An external attacker with network access to port 3000 can connect to the Socket.IO endpoint using a...

Severity:	High
Status:	Open
STRIDE:	Spoofing
Affected Assets:	CTF Flags
Identified:	August 13, 2026

Threat Source

External attacker (non-browser client)

Prerequisites

Attacker can reach the backend Socket.IO endpoint on port 3000, which is network-accessible; CORS origin restriction does not apply to non-browser clients

Threat Action

Connect to the Socket.IO endpoint using a custom script (bypassing CORS) and impersonate a legitimate training participant without any authentication challenge

Impact

Attacker can emit verification events and solve challenges on behalf of others, or receive all broadcast notifications including CTF flags

Evidence

- registerWebsocketEvents.ts registers io.on('connection') with no auth/token verification; posture confirms CORS bypass by non-browser clients.

Anchor

node: End User (Browser)

An authenticated end-user can store an attacker-controlled URL in the profileImage database column...

Severity:	High
Status:	Open
STRIDE:	Tampering
Affected Assets:	profile-image-data
Identified:	August 13, 2026

Threat Source

end-user-via-browser

Prerequisites

Authenticated session; supplied URL triggers a fetch failure so the raw URL is persisted

Threat Action

Store an attacker-controlled redirect URL (e.g., <https://evil.com/phish>) as profileImage value in the Users table, which the frontend later renders as an image src or link, redirecting victims to a phishing site

Impact

Phishing attacks against other users who view the attacker's profile, leveraging application trust

Evidence

- Posture explicitly notes 'On fetch failure, the raw user-supplied URL is persisted directly into the database as profileImage, enabling stored-XSS or open-redirect' — models/user.ts and routes/profile

Anchor

node: SQLite Users Table

An authenticated end-user can perform SSRF requests through the profile image URL handler without...

Severity:	High
Status:	Open
STRIDE:	Repudiation
Affected Assets:	internal-network-responses
Identified:	August 13, 2026

Threat Source

end-user-via-browser

Prerequisites

#Empty

Threat Action

Perform SSRF requests to internal services through the profile image upload endpoint without those outbound requests being logged with user attribution, making it impossible to trace which user probed which internal target

Impact

Inability to attribute internal network probing to a specific user session, hindering incident response

Evidence

- The profileImageUrlUpload.ts handler performs fetch() without logging the target URL or user identity; no audit trail is mentioned in posture or route handler code.

Anchor

node: Express Backend (profileImageUrlUpload)

A network-adjacent attacker on the same LAN can sniff unencrypted HTTP traffic carrying the JWT...

Severity:	High
Status:	Open
STRIDE:	Information disclosure
Affected Assets:	jwt-token
Identified:	August 13, 2026

Threat Source

network-adjacent attacker (same LAN or Wi-Fi)

Prerequisites

Default local deployment with no HTTPS enforcement; attacker on same network segment

Threat Action

Sniff HTTP traffic to capture the Authorization Bearer JWT, token cookie, and imageUrl payload from the POST request to /profile/image/url

Impact

Credential theft enabling session hijacking and arbitrary SSRF execution as the victim

Evidence

- Posture states 'No HTTPS enforcement in default local deployment' for Hop 1; request.interceptor.ts attaches JWT Bearer token per frontend key files.

Anchor

flow: Angular Frontend SPA -> Express Backend (profileImageUrlUpload)

An authenticated end-user controlling an external server can serve arbitrary malicious content...

Severity:	High
Status:	Open
STRIDE:	Tampering
Affected Assets:	profile-image-data
Identified:	August 13, 2026

Threat Source

end-user-via-browser controlling an external server

Prerequisites

Authenticated session; attacker controls the HTTP server at the supplied URL

Threat Action

Serve a malicious payload (e.g., web shell, HTML with scripts, or polyglot file) from the attacker-controlled server which the backend writes to the publicly-accessible uploads directory at `frontend/dist/frontend/assets/public/images/uploads/{userId}.{ext}`

Impact

Persistent malicious content served from the application's own origin, enabling phishing, drive-by-download, or same-origin script execution

Evidence

- The handler writes fetched response body to `frontend/dist/frontend/assets/public/images/uploads/{id}.{ext}` per `profileImageUrlUpload.ts` and posture's Profile Image Filesystem element description.

Anchor

node: Profile Image Filesystem

An external attacker with a stolen cookie token can invoke `/profile/image/url` without a valid JWT...

Severity:	High
Status:	Open
STRIDE:	Spoofing
Affected Assets:	jwt-token
Identified:	August 13, 2026

Threat Source

external attacker with stolen or forged cookie

Prerequisites

Attacker obtains a valid token cookie value (e.g., via XSS, network sniffing on unencrypted local deployment, or session fixation)

Threat Action

Send POST to `/profile/image/url` with only the stolen cookie — no JWT Authorization header needed — because the route is registered before `expressJwt` middleware in `server.ts`, bypassing the standard authentication layer

Impact

Attacker performs SSRF or profile image tampering as the victim user without possessing the victim's JWT

Evidence

- Posture notes the route is at line ~316 vs `isAuthorized()` at ~359 in `server.ts`; only cookie-based `authenticate-dUsers.get(req.cookies.token)` check applies.

Anchor

node: Express Backend (`profileImageUrlUpload`)

An authenticated end-user can supply a malicious URL that fails to fetch, causing the raw...

Severity:	High
Status:	Open
STRIDE:	Tampering
Affected Assets:	profile-image-data
Identified:	August 13, 2026

Threat Source

end-user-via-browser

Prerequisites

Authenticated session; the supplied URL must trigger a fetch failure (unreachable host, timeout, non-HTTP scheme)

Threat Action

Provide a malicious URL containing JavaScript payload (e.g., javascript: URI or data: URI with script) that is stored verbatim in the profileImage column of the SQLite Users table when the fetch fails

Impact

Stored cross-site scripting executing in other users' browsers when the profile image is rendered, enabling session hijacking or account takeover

Evidence

- Per posture: 'On fetch failure, the raw user-supplied URL is persisted directly into the database as profileImage, enabling stored-XSS or open-redirect' — routes/profileImageUrlUpload.ts and models/us

Anchor

node: profile-image-data

An authenticated end-user with a valid session cookie can supply a URL targeting internal network...

Severity:	High
Status:	Open
STRIDE:	Information disclosure
Affected Assets:	internal-network-responses
Identified:	August 13, 2026

Threat Source

end-user-via-browser

Prerequisites

Authenticated session with valid cookie token in authenticatedUsers map

Threat Action

Supply a URL pointing to internal network resources (e.g., `http://169.254.169.254/latest/meta-data/`, `http://localhost:3000/api/`) causing the backend to fetch and persist sensitive internal responses to the publicly-accessible uploads directory

Impact

Exfiltration of cloud metadata credentials, internal service data, or other sensitive information reachable from the backend's network position

Evidence

- The `profileImageUrlUpload.ts` handler calls `fetch(url)` with no URL validation or allowlist, as documented in `routes/profileImageUrlUpload.ts` and the SSRF boundary definition in `server.ts`.

Anchor

flow: Express Backend (`profileImageUrlUpload`) -> Arbitrary Remote Server (SSRF Target)

An end user, given that user messages are unfiltered and JWT identity is decoded without...

Severity:	High
Status:	Open
STRIDE:	Elevation of privilege
Affected Assets:	order-data
Identified:	August 13, 2026

Threat Source

End User

Prerequisites

No input sanitization on user messages; getOrderById tool queries orders by ID and relies on JWT-decoded userId for access control, but JWT is not verified

Threat Action

Craft adversarial messages that instruct the LLM to call getOrderById with order IDs belonging to other users, bypassing the intended access control since the forged or manipulated JWT identity is not verified

Impact

Unauthorized access to other users' order details (items, addresses, payment info) constituting a horizontal privilege escalation

Evidence

- The posture notes no input sanitization, JWT decoded without verification (lib/insecurity.ts), and getOrderById tool queries based on the unverified identity in routes/chat.ts.

Anchor

node: Chatbot Sub-process

An anonymous internet attacker, given that no rate limiting or authentication is applied to...

Severity:	High
Status:	Open
STRIDE:	Denial of service
Affected Assets:	llm-api-credentials
Identified:	August 13, 2026

Threat Source

Anonymous internet attacker

Prerequisites

No rate limiting applied to /rest/chat endpoint and no authentication required

Threat Action

Flood the /rest/chat endpoint with rapid unauthenticated requests, each of which triggers an expensive outbound LLM API call with streaming response handling

Impact

Exhaustion of backend resources (connection pool, memory for streaming responses) and LLM API quota/budget, degrading or denying service to legitimate users

Evidence

- The posture notes 'No rate limiting applied to /rest/chat endpoint' in the data flow description for Hop 1, and the endpoint requires no authentication (server.ts).

Anchor

flow: Frontend SPA -> Backend REST API

The external LLM API, given that no TLS enforcement is verified in source and the URL comes from...

Severity:	High
Status:	Open
STRIDE:	Information disclosure
Affected Assets:	llm-api-credentials
Identified:	August 13, 2026

Threat Source

External LLM API

Prerequisites

LLM_API_KEY is sent on every outbound request to the external LLM provider; no TLS enforcement verified in source code (URL comes from config)

Threat Action

If the LLM API URL is misconfigured to HTTP or an attacker-controlled endpoint, or if the external provider is compromised, the LLM_API_KEY and full system prompt with confidential policy are exposed

Impact

Compromise of LLM API credentials enabling unauthorized API usage at the organization's expense; disclosure of confidential business policies

Evidence

- The posture assumption states 'no TLS enforcement in source, URL comes from config' and identifies llm-api-credentials as sent on every outbound request (routes/chat.ts).

Anchor

flow: Chatbot Sub-process -> External LLM API

An anonymous internet attacker, given that security.decode uses jws.decode without signature...

Severity:	High
Status:	Open
STRIDE:	Spoofing
Affected Assets:	order-data
Identified:	August 13, 2026

Threat Source

Anonymous internet attacker

Prerequisites

The chatbot handler decodes JWT without signature verification via security.decode (jws.decode, not jws.verify) in lib/insecurity.ts, and the endpoint has no auth middleware

Threat Action

Forge a JWT with an arbitrary userId/email payload and submit it in the Authorization header to the /rest/chat endpoint; the chatbot decodes it without verification and uses the claimed identity for order lookups

Impact

Attacker can impersonate any user and access their order data via the getOrderById tool, constituting an IDOR vulnerability

Evidence

- The posture states 'The JWT is decoded without signature verification inside the chat handler (security.decode uses jws.decode, not jws.verify)' as documented in lib/insecurity.ts and routes/chat.ts.

Anchor

node: Chatbot Sub-process

An end user, given that the getProductReviews tool uses MongoDB \$where with string concatenation...

Severity:	High
Status:	Open
STRIDE:	Tampering
Affected Assets:	order-data
Identified:	August 13, 2026

Threat Source

End User

Prerequisites

The getProductReviews tool uses MongoDB \$where with string concatenation from LLM-controlled input (routes/chat.ts line ~130)

Threat Action

Craft chat messages that cause the LLM to pass malicious values to the getProductReviews tool parameter, which are concatenated into a \$where JavaScript expression enabling arbitrary code execution in the MarsDB query context

Impact

Arbitrary data extraction, modification, or deletion in the MarsDB reviews collection; potential server-side JavaScript execution

Evidence

- The posture explicitly notes 'The getProductReviews tool uses MongoDB \$where with string concatenation from LLM-controlled input, creating a NoSQL injection path' in routes/chat.ts.

Anchor

node: MarsDB Reviews Collection

An end user can craft chat messages that instruct the LLM to reveal its system prompt contents...

Severity:	High
Status:	Open
STRIDE:	Information disclosure
Affected Assets:	system-prompt-with-confidential-policy
Identified:	August 13, 2026

Threat Source

End User

Prerequisites

#Empty

Threat Action

Craft chat messages that instruct the LLM to reveal its system prompt contents, including the confidential 15% courtesy discount escalation policy marked 'CONFIDENTIAL - INTERNAL ONLY'

Impact

Disclosure of internal business logic and discount thresholds to unauthorized users, enabling social engineering or abuse of escalation policies

Evidence

- The posture states the system prompt contains coupon policy including a confidential 15% courtesy discount escalation policy marked 'CONFIDENTIAL - INTERNAL ONLY' and notes no prompt-injection filteri

Anchor

node: system-prompt-with-confidential-policy

An end user can craft adversarial chat messages that override the system prompt instructions,...

Severity:	High
Status:	Open
STRIDE:	Elevation of privilege
Affected Assets:	coupon-codes
Identified:	August 13, 2026

Threat Source

End User

Prerequisites

#Empty

Threat Action

Craft adversarial chat messages that override the system prompt instructions, causing the LLM to invoke the generate-Coupon tool with arbitrary discount percentages (e.g., 100% off)

Impact

Generation of high-value discount coupons beyond authorized policy limits, enabling financial fraud against the store

Evidence

- The posture identifies no input sanitization or prompt-injection filtering in routes/chat.ts. User messages from req.body.messages are passed directly to streamText alongside tool definitions includin

Anchor

node: Chatbot Sub-process

An anonymous internet attacker, given that the /rest/chat endpoint has no security.isAuthenticated()...

Severity:	High
Status:	Open
STRIDE:	Spoofing
Affected Assets:	coupon-codes
Identified:	August 13, 2026

Threat Source

Anonymous internet attacker

Prerequisites

The /rest/chat endpoint has no security.isAuthenticated() middleware, making it accessible without a valid JWT

Threat Action

Send HTTP POST requests directly to /rest/chat without any authentication token, impersonating any user or acting anonymously to invoke chatbot functionality

Impact

Unauthorized access to chatbot tools including coupon generation, product search, and order lookup without any identity verification

Evidence

- The posture notes /rest/chat has NO server-side authentication enforcement (no security.isAuthenticated() middleware in server.ts), confirmed in routes/chat.ts and server.ts.

Anchor

flow: Frontend SPA -> Backend REST API

An anonymous internet attacker, given that no authentication is enforced and no user identity is...

Severity:	High
Status:	Open
STRIDE:	Repudiation
Affected Assets:	Uploaded File Content
Identified:	August 13, 2026

Threat Source

Anonymous internet attacker

Prerequisites

No authentication is enforced on /file-upload so no user identity is captured; no dedicated logging of file upload source IP or content characteristics

Threat Action

Repeatedly exploit XXE, zip-slip, or YAML deserialization vulnerabilities without any authenticated identity being logged, making forensic attribution of the attack impossible

Impact

Security team cannot attribute file upload attacks to specific actors, hindering incident response and enabling persistent undetected exploitation

Evidence

- server.ts line 316 registers /file-upload without security.isAuthenticated(); posture confirms no authentication context is verified in the middleware chain.

Anchor

node: Backend Express Server

A network-positioned attacker, given that the application defaults to HTTP without TLS enforcement,...

Severity:	High
Status:	Open
STRIDE:	Information disclosure
Affected Assets:	JWT Bearer Token
Identified:	August 13, 2026

Threat Source

Network-positioned attacker (same network segment)

Prerequisites

Application defaults to HTTP without TLS enforcement (hostServer is http://localhost:3000); attacker is on the same network segment as the user

Threat Action

Intercept the multipart POST request to /file-upload containing the Bearer JWT token in the Authorization header via passive network sniffing or ARP spoofing on an unencrypted connection

Impact

Attacker obtains valid JWT tokens enabling session hijacking and access to authenticated endpoints across the application

Evidence

- complaint.component.ts attaches Bearer token in Authorization header; posture assumption states 'Transport is HTTP without TLS enforcement' based on hostServer being http://localhost:3000.

Anchor

node: JWT Bearer Token

An anonymous internet attacker, given that parsed XML with resolved entities is returned in the...

Severity:

High

Status:

Open

STRIDE:

Information disclosure

Affected Assets:

XML Parsed Output (with resolved entities)

Identified:

August 13, 2026

Threat Source

Anonymous internet attacker

Prerequisites

XML parsing resolves external entities and the parsed result is returned in the HTTP response body truncated to 400 characters

Threat Action

Craft XML with external entity references targeting specific sensitive files and iteratively exfiltrate their contents via the 400-character truncated error response, reconstructing full file contents across multiple requests

Impact

Systematic exfiltration of encryption keys, JWT private key, database credentials, and other secrets from the server filesystem through repeated requests

Evidence

- routes/fileUpload.ts returns parsed XML output via utils.trunc truncated to 400 chars; lib/xml.ts parseXmlString resolves external entities and returns the result string.

Anchor

node: XML Parsed Output (with resolved entities)

An anonymous internet attacker, given that checkFileType and checkUploadSize only log challenge...

Severity:	High
Status:	Open
STRIDE:	Tampering
Affected Assets:	Uploaded File Content
Identified:	August 13, 2026

Threat Source

Anonymous internet attacker

Prerequisites

checkFileType and checkUploadSize only log challenge solutions and do not reject uploads; no content sanitization is applied before parsing

Threat Action

Submit file content with arbitrary MIME type and malicious payload that passes through the middleware chain unmodified into the ZIP/XML/YAML parsers, exploiting the lack of content validation at the boundary between request handling and file processing

Impact

Attacker-controlled content reaches all three vulnerable parsers in sequence, maximizing exploitation surface — a single upload can trigger XXE, zip-slip, and deserialization attacks

Evidence

- routes/fileUpload.ts chains handleZipFileUpload, handleXmlUpload, handleYamlUpload in sequence; posture confirms checkUploadSize and checkFileType 'only log challenge solutions, they do not reject upl

Anchor

flow: Backend Express Server ' File Upload Processor

An anonymous internet attacker, given that YAML parsing uses js-yaml.load (not safeLoad) inside a...

Severity:	High
Status:	Open
STRIDE:	Elevation of privilege
Affected Assets:	Server Filesystem Data (XXE Exfiltration Target)
Identified:	August 13, 2026

Threat Source

Anonymous internet attacker

Prerequisites

Attacker can reach /file-upload without authentication;YAML parsing uses js-yaml.load (not safeLoad) inside a vm context

Threat Action

Upload a YAML document containing JavaScript type constructors (!!js/function or similar) that exploit js-yaml.load's unsafe deserialization to execute arbitrary code within the vm sandbox, potentially escaping via known vm breakout techniques

Impact

Attacker achieves arbitrary code execution on the backend server, gaining full control over the Node.js process and access to all server resources

Evidence

- routes/fileUpload.ts uses js-yaml.load (not safeLoad) in vm context as described in posture's File Upload Processor element definition.

Anchor

node: File Upload Processor

An anonymous internet attacker, given that ZIP extraction uses an insufficient path.resolve...

Severity:	High
Status:	Open
STRIDE:	Tampering
Affected Assets:	Extracted ZIP File Entries
Identified:	August 13, 2026

Threat Source

Anonymous internet attacker

Prerequisites

Attacker can reach /file-upload without authentication; ZIP extraction uses path.resolve check that only verifies absolutePath.includes(path.resolve('.')) which is insufficient for parent directory prefixes

Threat Action

Upload a specially crafted ZIP archive containing entries with directory traversal paths (e.g., ../../../server.ts) that pass the inadequate path.resolve includes check, causing files to be written outside the uploads/complaints/ directory

Impact

Attacker overwrites application source files, configuration, or dependencies on the server filesystem, achieving remote code execution or persistent backdoor installation

Evidence

- routes/fileUpload.ts extractZipBuffer uses path.resolve check that only verifies absolutePath.includes(path.resolve('.')) — documented as insufficient zip-slip prevention in trust boundary 'Parser to

Anchor

node: File Upload Processor

An anonymous internet attacker, given that the XML parser is configured with...

Severity:	High
Status:	Open
STRIDE:	Information disclosure
Affected Assets:	Server Filesystem Data (XXE Exfiltration Target)
Identified:	August 13, 2026

Threat Source

Anonymous internet attacker

Prerequisites

Attacker can reach /file-upload without authentication; XML parser configured with XML_PARSE_NOENT|XML_PARSE_DTDLOAD and xmlRegisterFsInputProviders() in lib/xml.ts

Threat Action

Upload an XML document containing a DTD with external entity references (e.g., <!ENTITY xxe SYSTEM 'file:///etc/passwd') causing libxml2-wasm to resolve entities from the server filesystem and return content in the parsed output

Impact

Sensitive server files including /etc/passwd, application encryption keys (encryptionkeys/ directory), and source code are disclosed to the attacker in the HTTP response body (truncated to 400 chars via utils.trunc)

Evidence

- lib/xml.ts configures XML_PARSE_NOENT|XML_PARSE_DTDLOAD and calls xmlRegisterFsInputProviders(); routes/fileUpload.ts returns parsed XML output truncated to 400 chars in error response.

Anchor

node: File Upload Processor

An anonymous internet attacker, given that the /file-upload endpoint has no server-side...

Severity:	High
Status:	Open
STRIDE:	Spoofing
Affected Assets:	Server Filesystem Data (XXE Exfiltration Target)
Identified:	August 13, 2026

Threat Source

Anonymous internet attacker

Prerequisites

The /file-upload endpoint has no server-side authentication enforcement (server.ts line 316 registers route without security.isAuthenticated() middleware)

Threat Action

Send a multipart POST directly to /file-upload without any valid JWT, bypassing all client-side checks and submitting a crafted XML file with external entity declarations

Impact

Attacker gains access to server-side file processing pipeline without any identity verification, enabling exploitation of XXE, zip-slip, or YAML deserialization

Evidence

- server.ts line 316 registers /file-upload without security.isAuthenticated() middleware; complaint.component.ts shows client-side token attachment that is not enforced server-side.

Anchor

flow: Frontend Complaint Component ' Backend Express Server

An attacker with database read access via SQL injection can perform offline rainbow-table attacks...

Severity:	High
Status:	Open
STRIDE:	Information disclosure
Affected Assets:	user-credentials
Identified:	August 13, 2026

Threat Source

attacker with database read access (via SQL injection or file access)

Prerequisites

Attacker obtains user records from SQLite via SQL injection (proven feasible per threat 1) which returns password hashes

Threat Action

perform offline rainbow-table or dictionary attack against the unsalted MD5 password hashes, recovering plaintext passwords in seconds for common passwords

Impact

mass credential compromise enabling account takeover across all users, and credential-stuffing attacks on external services where users reuse passwords

Evidence

- Posture documents password-hash as 'MD5 hash computed by security.hash() (crypto.createHash("md5")) in lib/insecurity.ts' with no salt, and SQL injection access to the database is established via rout

Anchor

node: password-hash

A network-level attacker in MITM position, given application uses HTTP without TLS, can modify the...

Severity:	High
Status:	Open
STRIDE:	Tampering
Affected Assets:	jwt-token
Identified:	August 13, 2026

Threat Source

network-level attacker (MITM position)

Prerequisites

Application uses HTTP without TLS (default configuration) and attacker occupies a man-in-the-middle position on the network path

Threat Action

modify the HTTP POST body in transit, changing the email field to a different user's address or injecting SQL payload, or altering the response to substitute a different JWT

Impact

attacker redirects login to a different account or injects attack payloads transparently, or substitutes a token granting attacker-controlled session to the victim

Evidence

- Hop 2 documents HTTP transport without TLS; posture's 'Browser to Backend Network Boundary' notes 'no TLS enforcement observed in source.'

Anchor

flow: Auth Flow (Frontend) ' REST API Login (Backend)

An external attacker, given no rate limiting or account lockout exists on /rest/user/login, can...

Severity:	High
Status:	Open
STRIDE:	Spoofing
Affected Assets:	user-credentials
Identified:	August 13, 2026

Threat Source

external attacker

Prerequisites

No rate limiting or account lockout mechanism on /rest/user/login endpoint as confirmed by posture assumptions

Threat Action

automate high-volume login attempts with common password dictionaries against known or enumerated email addresses

Impact

attacker discovers valid credentials for accounts with weak passwords (especially given MD5 hashing encourages short/common passwords), enabling unauthorized access

Evidence

- Posture assumption states 'No rate limiting or account lockout mechanism exists on the /rest/user/login endpoint' and passwords are stored as unsalted MD5 hashes per lib/insecurity.ts.

Anchor

node: REST API Login (Backend)

An attacker exploiting XSS in the Juice Shop frontend, given the token is stored in localStorage...

Severity:	High
Status:	Open
STRIDE:	Information disclosure
Affected Assets:	jwt-token
Identified:	August 13, 2026

Threat Source

attacker exploiting XSS in Juice Shop frontend

Prerequisites

A cross-site scripting vulnerability exists elsewhere in the application (Juice Shop contains multiple intentional XSS flaws) and token cookie lacks HttpOnly flag

Threat Action

execute JavaScript in the victim's browser context to read localStorage.getItem('token') and document.cookie, exfiltrating the JWT to an attacker-controlled server

Impact

attacker obtains victim's valid JWT enabling session hijacking and full account takeover for the token's 6-hour lifetime

Evidence

- Posture assumption notes 'cookie set by frontend does not set HttpOnly or Secure flags, making the token accessible to client-side JavaScript (XSS)' and login.component.ts stores token via localStorag

Anchor

node: Browser localStorage/Cookie

An external attacker with access to public source code, given the RSA private key is hardcoded in...

Severity:	High
Status:	Open
STRIDE:	Spoofing
Affected Assets:	jwt-token
Identified:	August 13, 2026

Threat Source

external attacker with access to public source code

Prerequisites

The RSA private key is hardcoded as a string constant in the open-source lib/insecurity.ts; attacker reads the repository

Threat Action

sign arbitrary JWTs with any user identity (id, email, role=admin) using the exposed private key, presenting them as valid Bearer tokens

Impact

attacker impersonates any user or escalates to admin role without authentication, bypassing the entire login flow

Evidence

- Posture documents jwt-signing-private-key as 'RSA private key hardcoded as a string constant in lib/insecurity.ts' with origin pre-existing, and the distinctive section confirms 'token forgery possible'

Anchor

node: jwt-signing-private-key

A network-level attacker, given application deployed with default HTTP configuration without TLS,...

Severity:	High
Status:	Open
STRIDE:	Information disclosure
Affected Assets:	user-credentials
Identified:	August 13, 2026

Threat Source

network-level attacker (same LAN or MITM position)

Prerequisites

Application deployed with default configuration using HTTP (environment.hostServer = http://localhost:3000) without TLS-terminating reverse proxy

Threat Action

passively sniff or actively intercept the plaintext HTTP POST containing email and password in the JSON body

Impact

attacker obtains valid user credentials enabling account takeover and credential reuse attacks

Evidence

- Posture assumption states backend serves over HTTP without TLS in default configuration; hop 2 explicitly notes 'Transport is HTTP (not HTTPS by default). Payload is plaintext credentials.'

Anchor

flow: Auth Flow (Frontend) ' REST API Login (Backend)

An end-user-via-browser can inject SQL payload into the email field which is interpolated directly...

Severity:	High
Status:	Open
STRIDE:	Tampering
Affected Assets:	user-credentials
Identified:	August 13, 2026

Threat Source

end-user-via-browser

Prerequisites

#Empty

Threat Action

inject SQL payload (e.g., ' OR 1=1--') into the email field which is interpolated directly into the raw SQL query in routes/login.ts, returning arbitrary user records

Impact

attacker authenticates as any user (including admin) without knowing valid credentials, gaining full account takeover

Evidence

- routes/login.ts uses unsanitized string interpolation of req.body.email into models.sequelize.query() as documented in posture's 'User Input to SQL Execution' trust boundary and hop 3.

Anchor

flow: REST API Login (Backend) ' SQLite User Database

An unresponsive external webhook receiver can cause the outbound fetch in lib/webhook.ts to hang or...

Severity:	Medium
Status:	Open
STRIDE:	Denial of service
Affected Assets:	challenge-solution-data
Identified:	August 13, 2026

Threat Source

External Webhook Receiver (or network conditions)

Prerequisites

SOLUTIONS_WEBHOOK is configured and the external endpoint becomes slow or unresponsive

Threat Action

Cause the backend's webhook notify() call to hang or timeout on every challenge solve, potentially blocking or slowing the Node.js event loop if fetch does not have a short timeout configured, degrading challenge-detection responsiveness for all users

Impact

Degraded backend performance or unhandled promise rejections disrupting the challenge solution experience for concurrent users

Evidence

- lib/webhook.ts uses fetch with no documented timeout; Node.js single-threaded event loop is sensitive to blocking outbound calls without timeout guards.

Anchor

node: Challenge Detection & Webhook Notifier

An external attacker who knows the webhook URL can send crafted POST requests to the external...

Severity:	Medium
Status:	Open
STRIDE:	Spoofing
Affected Assets:	challenge-solution-data
Identified:	August 13, 2026

Threat Source

External attacker with knowledge of webhook endpoint URL

Prerequisites

Attacker knows or discovers the SOLUTIONS_WEBHOOK URL (e.g., leaked in logs, config repos, or network traffic interception)

Threat Action

Send crafted POST requests mimicking the webhook payload format (challenge key, ctfFlag, cheat score, issuer block) directly to the external webhook receiver, since the outbound request carries no authentication headers or HMAC signature

Impact

The webhook receiver accepts fabricated challenge-solved events, corrupting CTF leaderboards, triggering false downstream integrations, or injecting bogus telemetry

Evidence

- Posture confirms no authentication or signing on outbound request (lib/webhook.ts); the receiver has no way to verify the sender's identity.

Anchor

node: External Webhook Receiver

An attacker with environment variable write access, given that webhook-url has no validation or...

Severity:	Medium
Status:	Open
STRIDE:	Elevation of privilege
Affected Assets:	webhook-url
Identified:	August 13, 2026

Threat Source

Attacker with environment variable write access

Prerequisites

Attacker gains ability to set or modify the SOLUTIONS_WEBHOOK environment variable (e.g., via container escape, CI/CD pipeline manipulation, or exploiting another vulnerability in the intentionally-vulnerable app that allows env modification)

Threat Action

Set SOLUTIONS_WEBHOOK to an internal network address (e.g., cloud metadata endpoint 169.254.169.254, internal microservice), causing the backend to POST challenge solution payloads to attacker-chosen internal targets

Impact

Server-side request forgery allowing access to internal services, cloud metadata credentials, or port-scanning of internal infrastructure

Evidence

- Posture notes webhook URL is sourced from process.env.SOLUTIONS_WEBHOOK with no validation or allowlisting (lib/webhook.ts); the distinctive section explicitly calls out SSRF potential.

Anchor

node: Challenge Detection & Webhook Notifier

An operator with Docker socket access can build and deploy a tampered container image and deny...

Severity:	Medium
Status:	Open
STRIDE:	Repudiation
Affected Assets:	container-image
Identified:	August 13, 2026

Threat Source

Operator (or anyone with Docker socket access)

Prerequisites

Multiple users share the operator workstation or Docker socket access is granted to a group; no container-level audit logging beyond host OS logs

Threat Action

Build and deploy a modified container image (e.g., with backdoor) and deny responsibility, since Docker daemon operations lack per-user attribution beyond OS-level socket access logs

Impact

Inability to attribute who deployed a malicious or misconfigured container, hindering incident response and accountability

Evidence

- The trust boundary 'Operator CLI to Docker Daemon' notes no additional authentication beyond OS-level Docker socket access per code_sc-b5c04c05-c24b-446e-ae94-9fbee71dc7a/juice-shop-master/infrastruc

Anchor

node: Operator

A malicious third-party website, exploiting the fully open CORS policy (app.options('*', cors())),...

Severity:	Medium
Status:	Open
STRIDE:	Information disclosure
Affected Assets:	Frontend JavaScript Bundle
Identified:	August 13, 2026

Threat Source

malicious third-party website

Prerequisites

User visits attacker-controlled website while having active Juice Shop session; CORS is fully open (app.options('*', cors()))

Threat Action

Issue cross-origin requests from attacker's site to the Juice Shop backend, leveraging the wildcard CORS policy to read responses including static assets and potentially authenticated API data via the user's cookies/session

Impact

Attacker's site can read Juice Shop responses cross-origin, potentially accessing user-specific data or leveraging the open policy in conjunction with authenticated endpoints

Evidence

- Posture states 'CORS is fully open (app.options('*', cors()))' at the Server-to-Browser Delivery Boundary; server.ts is a key file for the backend component.

Anchor

node: Express Static File Server

An anonymous end user can inject computationally expensive SQL statements through the...

Severity:	Medium
Status:	Open
STRIDE:	Denial of service
Affected Assets:	Product Data
Identified:	August 13, 2026

Threat Source

anonymous end user

Prerequisites

#Empty

Threat Action

inject computationally expensive SQL statements (e.g., nested subqueries, LIKE with wildcard-heavy patterns, or cross-joins) through the unauthenticated search endpoint to exhaust SQLite's single-writer lock and CPU

Impact

application becomes unresponsive for all users as the single-file SQLite database blocks on long-running injected queries

Evidence

- routes/search.ts allows arbitrary SQL injection with only a 200-character truncation; SQLite's architecture uses file-level locking (models/index.ts configures juiceshop.sqlite).

Anchor

node: SQLite Database

An authenticated end user holding any valid JWT can send POST/PUT requests to /api/BasketItems with...

Severity:	Medium
Status:	Open
STRIDE:	Elevation of privilege
Affected Assets:	Basket Items
Identified:	August 13, 2026

Threat Source

authenticated end user

Prerequisites

attacker holds any valid JWT (obtainable by registering or forging with the hardcoded private key)

Threat Action

send POST/PUT requests to /api/BasketItems with another user's BasketId to add or modify items in a victim's shopping basket

Impact

unauthorized manipulation of other users' basket contents, enabling order tampering

Evidence

- The posture notes 'basket manipulation challenge demonstrates authorization bypass' on BasketItems; routes/basketItems.ts and finale-rest expose CRUD without owner validation.

Anchor

node: Basket Items

An external attacker can open thousands of concurrent unauthenticated WebSocket connections, which...

Severity:	Medium
Status:	Open
STRIDE:	Denial of service
Affected Assets:	Challenge Solved Notifications
Identified:	August 13, 2026

Threat Source

External attacker with network access to port 3000

Prerequisites

No rate limiting or connection throttling on Socket.IO endpoint; no authentication prevents mass connections

Threat Action

Open thousands of concurrent WebSocket connections to the Socket.IO endpoint, exhausting server memory and file descriptors in the single-threaded Node.js process

Impact

Backend becomes unresponsive, preventing legitimate training participants from using the application

Evidence

- Posture confirms no authentication on Socket.IO connections (registerWebsocketEvents.ts); no rate limiting or connection cap is mentioned in the security controls.

Anchor

node: WebSocket Events Handler (Backend)

A malicious training participant can deny triggering challenge solves since no identity is bound to...

Severity:	Medium
Status:	Open
STRIDE:	Repudiation
Affected Assets:	Challenge Solved State
Identified:	August 13, 2026

Threat Source

Malicious training participant

Prerequisites

Participant has solved challenges via WebSocket verification (no authentication binds actions to identity)

Threat Action

Deny having triggered challenge solves since no identity is associated with the WebSocket connection and no audit log records which client emitted verification events

Impact

Inability to attribute challenge solves to specific participants in multi-user CTF scenarios, enabling disputes and cheating denial

Evidence

- Posture confirms no token, session, or identity verification on the WebSocket connection in registerWebsocketEvents.ts.

Anchor

node: WebSocket Events Handler (Backend)

A malicious WebSocket client can send malformed or oversized payloads through verification events,...

Severity:	Medium
Status:	Open
STRIDE:	Tampering
Affected Assets:	Challenge Solved State
Identified:	August 13, 2026

Threat Source

Malicious WebSocket client

Prerequisites

Attacker is connected to the Socket.IO endpoint (no authentication required)

Threat Action

Send malformed or excessively large payloads via 'verifyLocalXssChallenge' or 'verifySvgInjectionChallenge' events to trigger unexpected behavior in criteria evaluation functions that process user-supplied data

Impact

Potential server-side errors, regex denial-of-service via crafted SVG patterns, or unintended challenge state manipulation

Evidence

- registerWebsocketEvents.ts passes user-supplied data directly to criteria closures; posture confirms no input validation layer exists on the WebSocket handler.

Anchor

node: WebSocket Events Handler (Backend)

An authenticated end-user can craft a URL with a manipulated extension or path component, causing...

Severity:	Medium
Status:	Open
STRIDE:	Information disclosure
Affected Assets:	profile-image-data
Identified:	August 13, 2026

Threat Source

end-user-via-browser

Prerequisites

Authenticated session; the backend derives file extension from the URL or Content-Type without sanitization

Threat Action

Craft a URL whose path or Content-Type header tricks the backend into writing the fetched file outside the intended uploads directory or with a dangerous extension (e.g., .html, .js) that is then served by the static file middleware

Impact

Reading or overwriting application files, or serving attacker-controlled scripts from the application origin

Evidence

- The file is written as {userId}.{ext} in frontend/dist/frontend/assets/public/images/uploads/ per profileImageUrlUpload.ts; extension derivation logic is attacker-influenced.

Anchor

node: Profile Image Filesystem

An authenticated end-user can supply URLs pointing to extremely large or slow-streaming resources,...

Severity:	Medium
Status:	Open
STRIDE:	Denial of service
Affected Assets:	internal-network-responses
Identified:	August 13, 2026

Threat Source

end-user-via-browser

Prerequisites

Authenticated session; attacker controls the target URL or targets a slow/infinite-stream endpoint

Threat Action

Supply URLs to very large files or slow-drip servers causing the backend to exhaust memory, disk, or connection pool resources while streaming the response to the filesystem

Impact

Backend process hangs or crashes, disk fills up, denying service to other users

Evidence

- The fetch() call in profileImageUrlUpload.ts streams response body to filesystem with no size limit or timeout documented in the route handler.

Anchor

node: Express Backend (profileImageUrlUpload)

The external LLM API, given that full user message history is forwarded on every request, can...

Severity:	Medium
Status:	Open
STRIDE:	Information disclosure
Affected Assets:	user-chat-messages
Identified:	August 13, 2026

Threat Source

External LLM API

Prerequisites

Full user message history from req.body.messages is forwarded to the external LLM on every request

Threat Action

The external LLM provider retains, logs, or uses the full conversation history containing potentially sensitive user information (personal details, order inquiries, complaints) for model training or other purposes

Impact

Uncontrolled disclosure of user personal information and conversation content to a third-party service beyond the application's data governance boundary

Evidence

- The posture documents that user messages are 'passed unfiltered to the LLM as conversation context' in Hop 3 (routes/chat.ts) with no data minimization applied.

Anchor

flow: Chatbot Sub-process -> External LLM API

The external LLM API, given that the AI SDK executes tool calls without additional authorization...

Severity:	Medium
Status:	Open
STRIDE:	Tampering
Affected Assets:	coupon-codes
Identified:	August 13, 2026

Threat Source

External LLM API

Prerequisites

The AI SDK executes tool calls returned by the LLM without additional authorization checks beyond what the tool's execute function implements

Threat Action

The LLM emits incorrect or hallucinated tool-call instructions (e.g., generateCoupon with unintended discount values, or getOrderById with fabricated order IDs) that the backend executes blindly

Impact

Generation of unauthorized coupons or retrieval/exposure of unintended data due to non-deterministic LLM behavior, even without adversarial user input

Evidence

- The posture assumption states 'The AI SDK executes tool calls returned by the LLM without additional authorization checks beyond what the tool's execute function implements' as noted in routes/chat.ts

Anchor

node: Chatbot Sub-process

An anonymous internet attacker, given that ZIP extraction writes to the server filesystem with no...

Severity:	Medium
Status:	Open
STRIDE:	Denial of service
Affected Assets:	Extracted ZIP File Entries
Identified:	August 13, 2026

Threat Source

Anonymous internet attacker

Prerequisites

ZIP extraction writes to uploads/complaints/ with insufficient path traversal prevention; no file size limit enforced server-side for extracted entries

Threat Action

Upload a ZIP archive containing numerous large files or deeply nested directories that are extracted to the server filesystem, filling disk space or creating excessive inodes in the uploads/complaints/ directory

Impact

Server filesystem becomes unavailable due to disk exhaustion, causing denial of service for file uploads and potentially crashing the Node.js process or other services sharing the filesystem

Evidence

- routes/fileUpload.ts extractZipBuffer uses unzipper to write to uploads/complaints/; posture notes multer uploadToMemory has no observable server-side file size limit.

Anchor

node: Server Filesystem

An authenticated end user can modify the basket ID in sessionStorage or API requests to reference...

Severity:	Medium
Status:	Open
STRIDE:	Tampering
Affected Assets:	session-basket-id
Identified:	August 13, 2026

Threat Source

authenticated end user

Prerequisites

Basket ID is a numeric value stored in sessionStorage and included in subsequent API requests; attacker has a valid JWT from their own login

Threat Action

modify the basket ID value in sessionStorage or directly in API requests to reference another user's basket, bypassing ownership checks if the backend relies on the client-supplied bid

Impact

attacker views or modifies another user's shopping cart contents, potentially placing orders on their behalf or extracting item preferences

Evidence

- Posture identifies session-basket-id stored in browser sessionStorage (key 'bid') in login.component.ts, noting 'Integrity matters — tampering could associate another user's basket.'

Anchor

node: session-basket-id

An end user or attacker using compromised credentials can perform account actions and later deny...

Severity:	Medium
Status:	Open
STRIDE:	Repudiation
Affected Assets:	user-credentials
Identified:	August 13, 2026

Threat Source

End User (or attacker using compromised credentials)

Prerequisites

No tamper-evident audit log of login attempts is referenced in the login route; lastLoginIp is stored but not in an append-only log

Threat Action

perform account actions (purchases, data changes) and later deny having logged in, as the system lacks a cryptographically-signed or append-only audit trail of authentication events

Impact

inability to attribute malicious actions to a specific actor, undermining fraud investigation and accountability

Evidence

- routes/login.ts updates lastLoginIp on the user record but posture identifies no append-only log store; the SQLite User Database element is not marked is_log_store.

Anchor

node: End User